실무 이펙티브 자바 Day 2

타입 안정성과 모던 자바(Java 8) 스타일

생존코딩 오준석

어제 SCV가 얻은 것, 오늘 막을 실수

어제는 SCV 한 기를 고치며 객체를 안전하게 만드는 법을 모았다.

어제의 문제	처방	남은 코드
SCV는 생체인가, 기계인가	역할은 인터페이스로 (아이템 20)	`Healable`, `Repairable`
체력 계산식이 세 군데에	상속 대신 컴포지션 (아이템 18)	`Health`
체력 1인 SCV를 시험하라	의존 객체 주입, 정적 팩토리 (아이템 5, 1)	`SCV.ofDamaged(1)`
집결지 순간이동 버그	방어적 복사, 불변 객체 (아이템 50, 17)	`Position`, 불변 `Health`

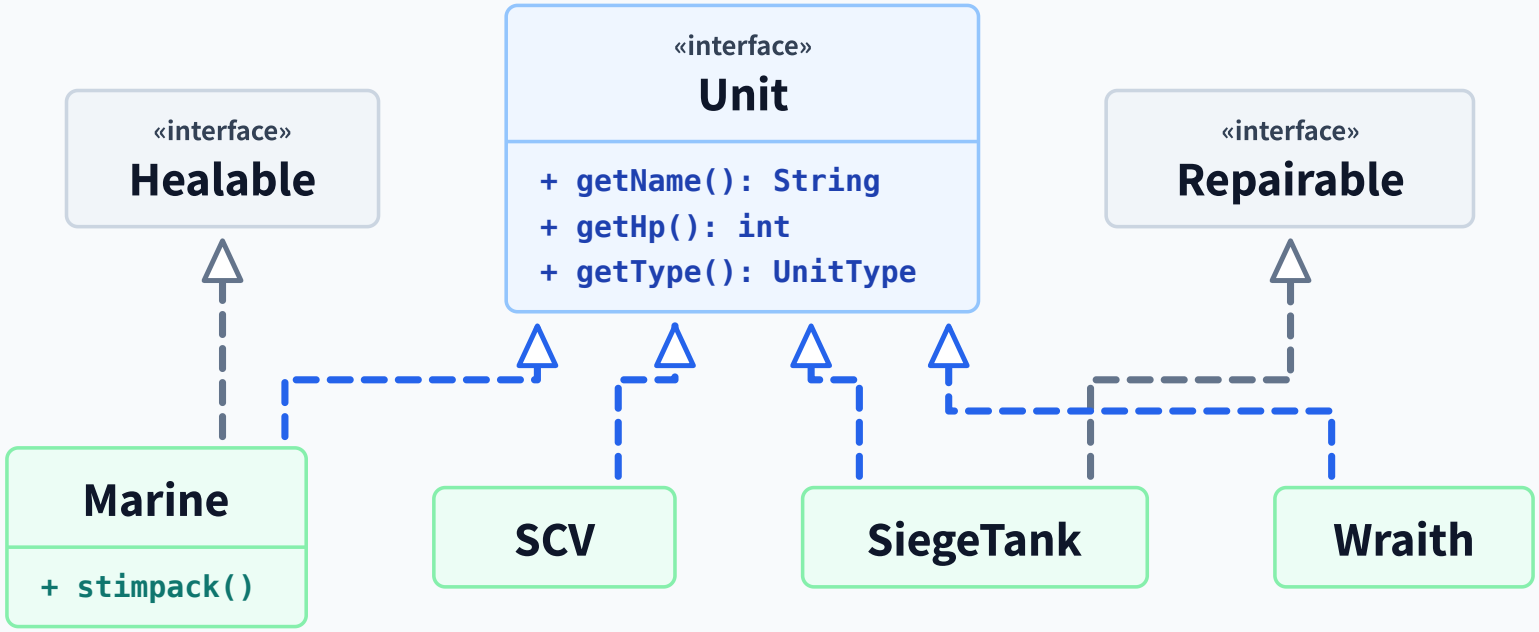
오후에는 빌더, try-with-resources, equals/hashCode, Comparable까지 붙였다.

> 오늘의 질문: 실수를 **실행하기 전에** 컴파일러가 잡게 할 수 있을까? 그리고 같은 일을 더 짧고 정확하게 쓸 수 있을까?

오늘의 흐름: 컴파일러가 잡게, 짧게, 정확하게

교시	오늘의 질문	묶음
1	꺼낼 때마다 캐스팅하는 벙커, 넣을 때 막을 수는 없나?	컴파일러가 잡게
2	마린 목록을 왜 공통 유틸 메서드에 못 넘기나?	컴파일러가 잡게
3	경고 없는 유틸, 타입별 보관함은 어떻게 만드나?	컴파일러가 잡게
4	조건 하나 바꾸려고 인터페이스를 또 만들어야 하나?	짧게
5	스트림처럼 생긴 반복문은 무엇이 문제인가?	짧게
6	`if (list != null)`은 언제까지 써야 하나?	짧게
7	정산 금액 1원은 어디로 사라졌나?	정확하게
8	동시에 담은 주문은 왜 사라지나?	정확하게

모든 유닛을 한 목록에: Unit 역할을 하나 더 붙인다



어제 유닛들에는 공통 타입이 없었다. 어제처럼 역할(인터페이스)을 하나 더 붙인다. 기존 역할은 그대로 둔다(SCV의 두 역할은 그림에서 생략).

Unit과 UnitType 코드: 어제 본 추상 클래스 Unit과는 다르다

어제 1교시 before의 `Unit`은 `hp`를 물려주던 추상 클래스였다. 오늘 `Unit`은 상태 없는 역할이다.

Unit.java

```
interface Unit {
    String getName();
    int getHp();
    UnitType getType();

    default boolean isAirUnit() {
        return getType() == UnitType.AIRCRAFT;
    }
}
```

UnitType.java

```
enum UnitType { WORKER, INFANTRY, VEHICLE, AIRCRAFT }
```

생체/기계로 나누지 않는다. SCV가 둘 다라서 분류가 모순된다. 모든 유닛은 `X.create()`, `X.ofDamaged(int)`로 만든다.

준비 코드 (1/6): Marine.java

오늘은 새 폴더에서 시작한다. 어제 만든 `Health.java`, `Position.java`, `Healable.java`, `Repairable.java`, `Book.java`만 그대로 가져오고, 유닛 클래스는 이 장부터 6장의 코드를 붙여 넣는다.

Marine.java

```
class Marine implements Unit, Healable {
    private static final int MAX_HP = 40;
    static final int STIMPACK_COST = 10;
    private Health health;

    Marine(Health health) { this.health = health; }
    static Marine create() { return new Marine(new Health(MAX_HP)); }
    static Marine ofDamaged(int currentHp) { return new Marine(new Health(currentHp, MAX_HP)); }

    @Override public String getName() { return "Marine"; }
    @Override public UnitType getType() { return UnitType.INFANTRY; }
    @Override public int getHp() { return health.getHp(); }
    @Override public void heal(int amount) { health = health.restore(amount); }
    void stimpack() { health = health.damage(STIMPACK_COST); }
}
```

어제와 달라진 곳: `implements Unit`, `getName()` · `getType()`, 스팀팩(체력 10 소모). 어제 테스트는 옛 생성자를 부르므로 가져오지 않는다.

준비 코드 (2/6): SCV.java

SCV.java

```java
import java.util.Objects;

class SCV implements Unit, Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;
    private Health health;
    private Position position;

    SCV(Health health, Position position) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
        this.position = Objects.requireNonNull(position);
    }
    static SCV create() { return new SCV(new Health(MAX_HP), new Position(0, 0)); }
    static SCV ofDamaged(int currentHp) { return new SCV(new Health(currentHp, MAX_HP), new Position(0, 0)); }
    static SCV at(Position position) { return new SCV(new Health(MAX_HP), position); }

    @Override public String getName() { return "SCV"; }
    @Override public UnitType getType() { return UnitType.WORKER; }
    @Override public int getHp() { return health.getHp(); }
    @Override public void heal(int amount) { health = health.restore(amount); }
    @Override public void repair(int amount) { health = health.restore(amount); }

    void repairTarget(Repairable target) { target.repair(REPAIR_AMOUNT); }
    Position getPosition() { return position; }
    void moveTo(Position target) { position = Objects.requireNonNull(target); }
    void moveBy(int dx, int dy) { position = position.move(dx, dy); }
}
```

어제 4교시의 SCV에 `Unit`의 세 메서드를 붙였다. 체력 60, 기본 위치 (0, 0).

준비 코드 (3/6): SiegeTank.java

SiegeTank.java

```
class SiegeTank implements Unit, Repairable {
    private static final int MAX_HP = 150;
    private Health health;

    SiegeTank(Health health) { this.health = health; }
    static SiegeTank create() { return new SiegeTank(new Health(MAX_HP)); }
    static SiegeTank ofDamaged(int currentHp) { return new SiegeTank(new Health(currentHp, MAX_HP)); }

    @Override public String getName() { return "SiegeTank"; }
    @Override public UnitType getType() { return UnitType.VEHICLE; }
    @Override public int getHp() { return health.getHp(); }
    @Override public void repair(int amount) { health = health.restore(amount); }
}
```

차량 유닛이라 SCV가 수리할 수 있다(`Repairable`). 체력 150.

준비 코드 (4/6): Wraith.java

Wraith.java

```
final class Wraith implements Unit {
    private static final int MAX_HP = 120;
    private Health health;

    Wraith(Health health) { this.health = health; }
    static Wraith create() { return new Wraith(new Health(MAX_HP)); }
    static Wraith ofDamaged(int currentHp) { return new Wraith(new Health(currentHp, MAX_HP)); }

    @Override public String getName() { return "Wraith"; }
    @Override public UnitType getType() { return UnitType.AIRCRAFT; }
    @Override public int getHp() { return health.getHp(); }
}
```

오늘 처음 나오는 공중 유닛. 체력 120, `isAirUnit()` 이 `true` 다.

준비 코드 (5/6): Medic.java

Medic.java

```
class Medic implements Unit, Healable {
    private static final int MAX_HP = 60;
    static final int HEAL_AMOUNT = 10;
    private Health health;

    Medic(Health health) { this.health = health; }
    static Medic create() { return new Medic(new Health(MAX_HP)); }
    static Medic ofDamaged(int currentHp) { return new Medic(new Health(currentHp, MAX_HP)); }

    @Override public String getName() { return "Medic"; }
    @Override public UnitType getType() { return UnitType.INFANTRY; }
    @Override public int getHp() { return health.getHp(); }
    @Override public void heal(int amount) { health = health.restore(amount); }
    void treat(Healable target) { target.heal(HEAL_AMOUNT); }
}
```

어제는 `treat`만 있던 메딕에 체력과 `Unit` 역할을 붙였다. 6교시에 쓴다.

준비 코드 (6/6): Vulture.java

Vulture.java

```
class Vulture implements Unit, Repairable {
    private static final int MAX_HP = 80;
    private Health health;

    Vulture(Health health) { this.health = health; }
    static Vulture create() { return new Vulture(new Health(MAX_HP)); }
    static Vulture ofDamaged(int currentHp) { return new Vulture(new Health(currentHp, MAX_HP)); }

    @Override public String getName() { return "Vulture"; }
    @Override public UnitType getType() { return UnitType.VEHICLE; }
    @Override public int getHp() { return health.getHp(); }
    @Override public void repair(int amount) { health = health.restore(amount); }
}
```

차량 유닛. 체력 80. 6교시에 쓴다.

1교시

꺼낼 때마다 캐스팅하는 벙커

게임 규칙상 벙커에는 보병만 들어간다.

Ver.1 벙커: 무엇이든 Object로 받는다

벙커에 유닛을 넣고(`enter`), 칸 번호로 꺼낸다(`get`). 어떤 유닛이든 받도록 `Object` 로 만들었다.

Bunker.java

```
import java.util.ArrayList;
import java.util.List;

class Bunker {
    private final List<Object> units = new ArrayList<>();

    void enter(Object unit) { units.add(unit); }
    Object get(int index) { return units.get(index); }
    int size() { return units.size(); }
}
```

- 꺼내면 `Object` 가 나온다. 마린으로 쓰려면 꺼낼 때마다 `(Marine)` 으로 캐스팅한다.
- `enter` 가 `Object` 를 받으니, 마린이든 시즈 탱크든 컴파일은 통과한다.

실수로 탱크를 넣으면, 꺼내는 줄에서 ClassCastException

BunkerTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class BunkerTest {

    @Test
    @DisplayName("벙커 두 번째 칸의 마린은 체력 40이다")
    void secondMarineInBunker() {
        Bunker bunker = new Bunker();
        bunker.enter(Marine.create());
        bunker.enter(SiegeTank.create()); // 실수: 탱크를 넣었다

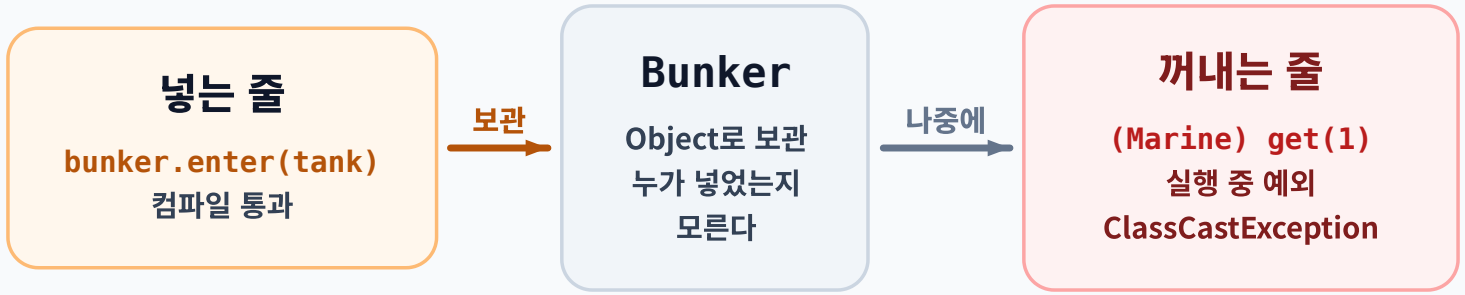
        Marine second = (Marine) bunker.get(1);

        assertEquals(40, second.getHp());
    }
}
```

```
java.lang.ClassCastException: class SiegeTank cannot be cast to class Marine (...)
    at BunkerTest.secondMarineInBunker(BunkerTest.java:15)
```

컴파일러는 아무 말도 하지 않았다. JDK 21 메시지이며, Java 8에서는 `SiegeTank cannot be cast to Marine`으로 짧게 나온다.

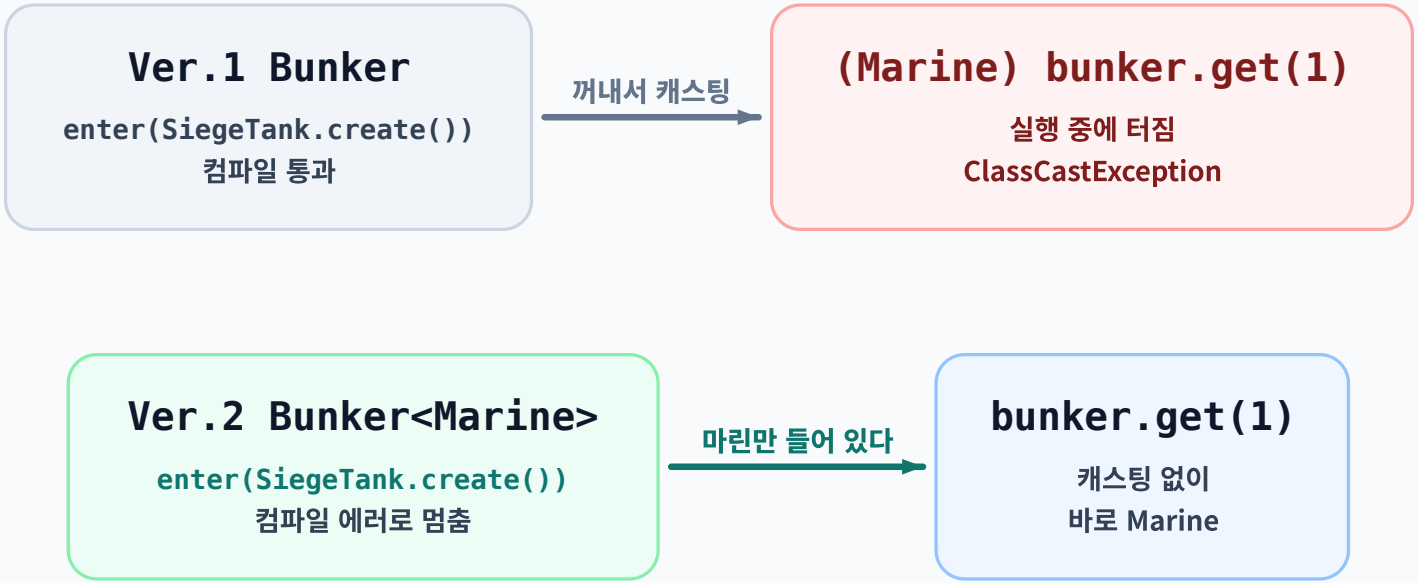
넣은 곳과 터진 곳이 멀다



- 잘못은 `enter` 줄에서 했는데, 예외는 꺼내서 캐스팅하는 줄에서 난다.
- 테스트에서는 두 줄이 붙어 있지만, 실제 코드에서는 넣는 곳과 꺼내는 곳이 다른 메서드, 다른 파일이다.
- 스택 트레이스는 **터진 곳**만 가리킨다. 탱크를 누가 넣었는지는 벙커를 거쳐 간 코드를 모두 뒤져야 안다.

> 넣는 그 줄에서 막을 수는 없을까?

쓰는 쪽이 타입을 정한다: 제네릭



`Bunker<T>`의 `T`(**타입 매개변수**)는 빈칸이다. 만들 때가 아니라 **쓰는 쪽이** 채운다. 이것이 **제네릭**이다.

> 비유: 입구에 "마린 전용" 팻말을 단 벙커는 탱크를 꺼낼 때가 아니라 입구에서 돌려보낸다.

Ver.2 벙커: Object 자리에 T를 쓴다

`Bunker.java`를 이것으로 바꾼다. 클래스 이름 옆에 `<T>`를 달고, `Object` 세 자리를 `T`로 바꿨다.

Bunker.java

```
import java.util.ArrayList;
import java.util.List;

class Bunker<T> {
    private final List<T> units = new ArrayList<>();

    void enter(T unit) { units.add(unit); }
    T get(int index) { return units.get(index); }
    int size() { return units.size(); }
}
```

- `Bunker<Marine>`이면 `enter`는 `Marine`만 받고, `get`은 `Marine`을 돌려준다.
- `Bunker<SCV>`, `Bunker<Wraith>`도 이 클래스 하나로 된다. 타입은 쓰는 쪽이 정한다.

Bunker<Marine>에 탱크를 넣으면 컴파일되지 않는다

`BunkerTest.java`의 `secondMarineInBunker()`를 이것으로 바꾼다. `(Marine)` 캐스팅이 사라졌다.

```
    @Test
    @DisplayName("벙커 두 번째 칸의 마린은 체력 40이다")
    void secondMarineInBunker() {
        Bunker<Marine> bunker = new Bunker<>();
        bunker.enter(Marine.create());
        bunker.enter(Marine.create());

        Marine second = bunker.get(1);

        assertEquals(40, second.getHp());
    }
```

아까의 탱크 줄을 다시 넣어 보면, 실행하기 전에 멈춘다.

```
bunker.enter(SiegeTank.create());
// 컴파일 에러: incompatible types: SiegeTank cannot be converted to Marine
```

탱크 줄을 지우고 실행하면 통과한다(체력 40).

벙커 전체 체력을 알고 싶다: T에게 getHp()를 부를 수 없다

`Bunker<T>`에 `totalHp()`를 붙여 보면 컴파일되지 않는다.

```
    int totalHp() {
        int total = 0;
        for (T unit : units) {
            total += unit.getHp();
        }
        return total;
    }
```

```
Bunker.java:14: error: cannot find symbol
  symbol:   method getHp()
  location: variable unit of type T
```

- `T`에는 무엇이든 올 수 있다. `Bunker<String>`도 되니, 컴파일러는 `getHp()`가 있다고 장담하지 못한다.
- "`T`는 `Unit`이다"라는 약속을 클래스에 적을 수 없을까?

Ver.3 벙커: T extends Unit으로 약속을 적는다

`Bunker.java`를 이것으로 바꾼다. 이제 `T`에는 `Unit`을 구현한 타입만 올 수 있다.

Bunker.java

```java
import java.util.ArrayList;
import java.util.List;

class Bunker<T extends Unit> {
    private final List<T> units = new ArrayList<>();

    void enter(T unit) { units.add(unit); }
    T get(int index) { return units.get(index); }
    int size() { return units.size(); }

    int totalHp() {
        int total = 0;
        for (T unit : units) {
            total += unit.getHp(); // T는 Unit이니 getHp()가 있다
        }
        return total;
    }
}
```

`BunkerTest.java`에 테스트를 하나 더 붙인다.

```java
    @Test
    @DisplayName("마린 2기가 들어간 벙커의 체력 합은 80이다")
    void totalHpOfBunker() {
        Bunker<Marine> bunker = new Bunker<>();
        bunker.enter(Marine.create());
        bunker.enter(Marine.create());

        assertEquals(80, bunker.totalHp());
    }
```

두 테스트 모두 통과한다. `Unit`이 아닌 타입은 막힌다.

```java
Bunker<String> names = new Bunker<String>();
// 컴파일 에러: type argument String
// is not within bounds of type-variable T
```

언제 제네릭을 쓰나

상황	이렇게 쓴다	예
담아 두었다 돌려주는 클래스	`<T>`	`Bunker<T>`, 현업의 `ApiResponse<T>`, `Page<T>`
담은 것의 기능을 불러야 하는 클래스	`<T extends Unit>`	`totalHp()`가 `getHp()`를 부른다
이미 있는 `List`·`Map`을 쓸 때	항상 원소 타입을 적는다	`List<Unit>`, `Map<String, Marine>`

> 기본은 `<T>`. 담은 것의 메서드가 필요할 때만 `extends`로 범위를 좁힌다. `Object`로 받는 Ver.1은 고르지 않는다.

JDK의 List도 Ver.2 벙커다: Raw 타입은 쓰지 않는다

구분	벙커 Ver.2	JDK `List<E>`
넣기	`void enter(T unit)`	`boolean add(E e)`
꺼내기	`T get(int index)`	`E get(int index)`
`<>`를 빼면	Ver.1 벙커(`Object`)	`List` — **Raw 타입**

`<>`를 빼고 `List`라고만 쓰면 Ver.1 벙커로 돌아간다. 아무거나 들어가고, 꺼낼 때마다 캐스팅하고, 틀리면 실행 중에 터진다. 이펙티브 자바 아이템 26: Raw 타입은 사용하지 말라.

> 현업 장면: iBATIS `queryForList()`가 돌려준 `List`에서 `(OrderVo) list.get(i)`로 꺼낼 때마다 캐스팅하던 코드. 그 `List`가 Raw 타입이다.

`javac -Xlint:rawtypes`로 컴파일하면 Raw 타입을 쓴 곳마다 `found raw type: List` 경고가 나온다.

배열은 실행해야 알고, List는 컴파일 때 안다

벙커를 배열로 만들었다면? 마린 배열을 `Unit[]` 변수로 받고 탱크를 넣어 본다.

UnitArrayTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class UnitArrayTest {

    @Test
    @DisplayName("유닛 배열 첫 칸에 시즈 탱크를 넣으면 꺼낼 수 있다")
    void putTankIntoUnitArray() {
        Unit[] units = new Marine[1];

        units[0] = SiegeTank.create();

        assertEquals("SiegeTank", units[0].getName());
    }
}
```

```
java.lang.ArrayStoreException: SiegeTank
    at UnitArrayTest.putTankIntoUnitArray(UnitArrayTest.java:13)
```

같은 일을 `List` 로 하면:

```java
List<Unit> list = new ArrayList<Marine>();
// 컴파일 에러: incompatible types:
// ArrayList<Marine> cannot be converted to List<Unit>
```

- 배열: 컴파일은 통과, **실행해야** `ArrayStoreException` 으로 안다.
- `List` : 대입하는 줄에서 **컴파일러가** 막는다.
- 섞어 담을 목록은 처음부터 `List<Unit>` 으로 만든다.
- 이펙티브 자바 아이템 28: 배열보다는 리스트를 사용하라.

책은 이 차이를 공변/불공변이라 부른다.

Ver.3 벙커에서 컴파일 에러인 줄은? (퀴즈)

퀴즈

`class Bunker<T extends Unit>` 기준이다. 컴파일 에러인 줄을 모두 고르면?

```
Bunker<Marine> marines = new Bunker<Marine>(); // 1
marines.enter(SiegeTank.create());             // 2
Marine m = marines.get(0);                     // 3
Bunker<String> names = new Bunker<String>();   // 4
```

1. 2만
2. 2와 4
3. 3과 4
4. 2, 3, 4

풀이: 2와 4가 컴파일 에러다

정답

정답은 2번(2와 4)이다.

줄	결과	이유
1	통과	`Marine`은 `Unit`이라 `T` 자리에 올 수 있다
2	에러	`Bunker<Marine>`의 `enter`는 `Marine`만 받는다
3	통과	`get`이 `Marine`을 돌려주니 캐스팅이 필요 없다
4	에러	`String`은 `Unit`이 아니라 `T extends Unit` 약속을 어긴다

> 2번 메시지: `SiegeTank cannot be converted to Marine`. 4번 메시지: `type argument String is not within bounds of type-variable T`. 둘 다 실행하기 전에 멈춘다.

섞어 담는 Squad를 처음부터 List<Unit>으로 만든다 (실습)

실습 마린, SCV, 탱크가 한 부대에 섞인다. 원소 타입은 공통 타입 `Unit`으로 적는다.

SquadTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadTest {

    @Test
    @DisplayName("마린 2 + SCV 1 부대의 체력 합은 140")
    void totalHpOfMixedSquad() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(Marine.create());
        squad.add(SCV.create());

        int total = squad.totalHp();

        assertEquals(140, total);
    }
}
```

Squad.java 시작 코드

```
import java.util.ArrayList;
import java.util.List;

class Squad {
    // 마린, SCV, 탱크를 섞어 담을 목록 필드를 여기에

    void add(Unit unit) { }
    List<Unit> getUnits() { return new ArrayList<>(); }
    int size() { return 0; }
    int totalHp() { return 0; }
}
```

1. 실행: `expected: <140> but was: <0>`
2. `Unit` 목록 필드를 만들고 네 메서드를 채운다 (`getUnits()`는 복사본).
3. 다시 실행해 통과를 확인한다.

실습 풀이: 섞어 담는 목록의 원소 타입은 Unit

정답

`Squad.java` 풀이

```
import java.util.ArrayList;
import java.util.List;

class Squad {
    private final List<Unit> units = new ArrayList<>();

    void add(Unit unit) { units.add(unit); }
    List<Unit> getUnits() { return new ArrayList<>(units); }
    int size() { return units.size(); }

    int totalHp() {
        int total = 0;
        for (Unit unit : units) {
            total += unit.getHp(); // 40 + 40 + 60 = 140
        }
        return total;
    }
}
```

이 `Squad` 에 `addAll(Collection<Unit>)` 을 하나 붙여 2교시에서 이어 간다.

2교시

List<Marine>을 못 받는 유틸 메서드

1교시에 만든 제네릭 목록을 공통 유틸에 넘겨 보자.

부대 체력 합계 유틸: totalHp(Collection<Unit>)

여러 화면이 "유닛 목록의 체력 합"을 구한다. 공통 유틸로 뽑았다.

Units.java

```
import java.util.Collection;

final class Units {
    private Units() {
    }

    static int totalHp(Collection<Unit> units) {
        int total = 0;
        for (Unit unit : units) {
            total += unit.getHp();
        }
        return total;
    }
}
```

확인(발췌)

```
List<Unit> units = new ArrayList<>();
units.add(Marine.create());
units.add(SiegeTank.create());

assertEquals(190, Units.totalHp(units));
// 40 + 150, 통과
```

마린 목록을 넘기면 컴파일 에러

마린만 모은 목록도 유닛 목록이다. 그런데 넘기지 못한다.

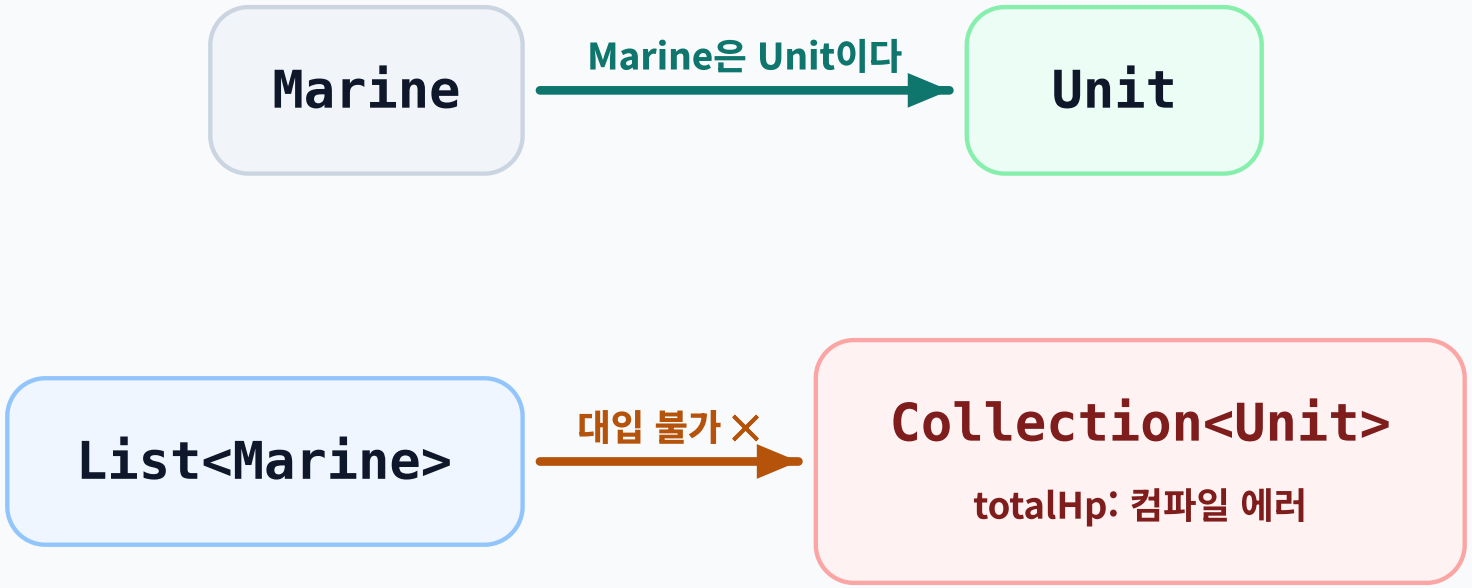
```
List<Marine> marines = new ArrayList<>();
marines.add(Marine.create());

int total = Units.totalHp(marines);
// 컴파일 에러: incompatible types: List<Marine> cannot be converted to Collection<Unit>
```

- `Marine`은 `Unit`인데, `List<Marine>`은 `Collection<Unit>` 자리에 못 들어간다.
- 호출부에서 `new ArrayList<Unit>(marines)`로 복사해 넘기는 우회 코드가 생긴다.

> 현업 장면: 공통 유틸 메서드가 `List<Object>`만 받아서 `List<OrderDto>`를 못 넘기고, 호출할 때마다 목록을 복사하던 코드와 같다.

List<Marine>은 List<Unit>이 아니다



- 마린은 유닛이지만, 마린 목록은 유닛 목록이 아니다. 1교시 배열 장의 `List<Unit> list = new ArrayList<Marine>()` 이 컴파일 에러였던 것과 같다.
- 허용했다면 `totalHp` 안의 `add` 로 마린 목록에 탱크가 들어간다. 컴파일러는 그 길을 막았다.

버티는 법: 유닛마다 메서드를 하나씩 복사한다

`Units.java`에 복사해 붙인 메서드들(발췌, 따라 치지 않는다):

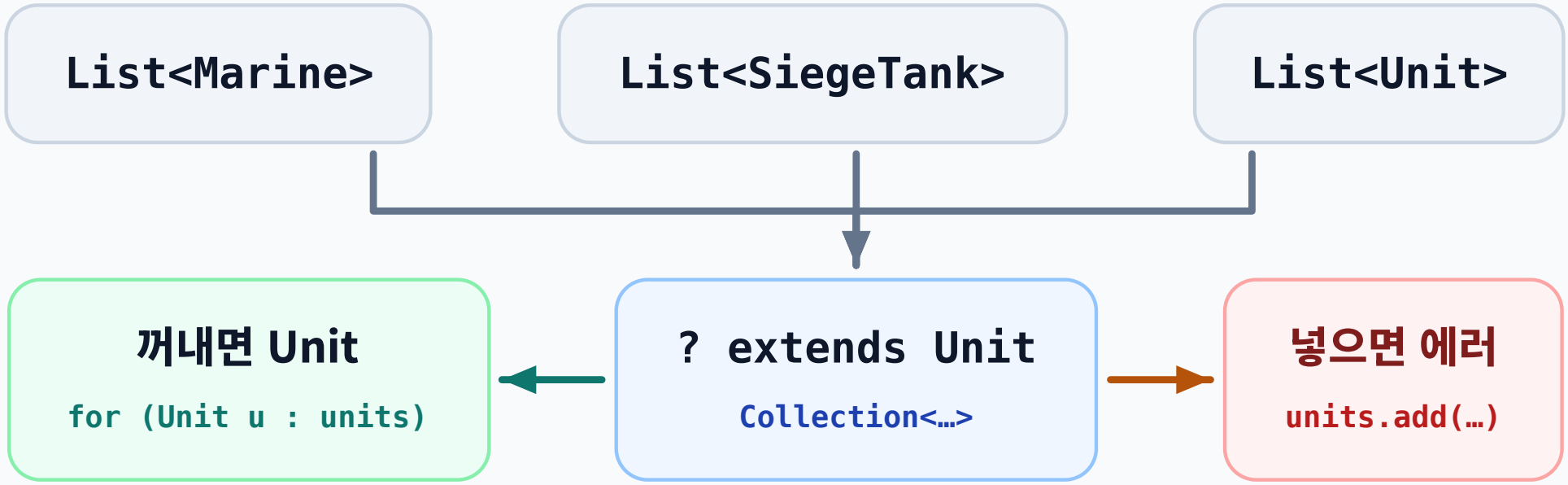
```
static int totalHpOfMarines(Collection<Marine> marines) {
    int total = 0;
    for (Marine marine : marines) {
        total += marine.getHp();
    }
    return total;
}

static int totalHpOfTanks(Collection<SiegeTank> tanks) {
    int total = 0;
    for (SiegeTank tank : tanks) {
        total += tank.getHp();
    }
    return total;
}

static int totalHpOfWraiths(Collection<Wraith> wraiths) { /* 또 같은 반복문 */ }
```

몸통은 한 글자도 다르지 않다. 유닛이 늘 때마다 메서드가 는다. "Unit이거나 그 하위 타입의 목록"이라고 적을 수는 없을까?

'Unit이거나 그 하위 타입'의 목록



`? extends Unit` 매개변수는 메서드 안에서 **읽기 전용**이다. 꺼내면 언제나 `Unit`이다. 아이템 31: 와일드카드로 API를 유연하게 만든다.

totalHp(Collection<? extends Unit>) 구현

앞에서 만든 `Units.java`의 매개변수 한 곳을 고친다. Units.java

```
import java.util.Collection;

final class Units {
    private Units() {
    }

    // 꺼내 쓰기만 한다 → ? extends Unit
    static int totalHp(Collection<? extends Unit> units) {
        int total = 0;
        for (Unit unit : units) { // 꺼낸 원소는 언제나 Unit
            total += unit.getHp();
        }
        return total;
    }
}
```

바뀐 곳은 매개변수 선언 한 곳이다. 몸통도, 호출부도 그대로다. 복사한 `totalHpOfXxx` 세 개는 지운다.

테스트: 마린 목록도, 탱크 목록도 받는다

UnitsTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class UnitsTest {

    @Test
    @DisplayName("마린 목록도, 탱크 목록도, 섞은 목록도 totalHp 하나로 계산한다")
    void totalHpAcceptsSubtypeLists() {
        List<Marine> marines = new ArrayList<>();
        marines.add(Marine.create());
        marines.add(Marine.create());
        List<SiegeTank> tanks = new ArrayList<>();
        tanks.add(SiegeTank.create());
        List<Unit> mixed = new ArrayList<>();
        mixed.add(Marine.create());
        mixed.add(SiegeTank.create());

        assertEquals(80, Units.totalHp(marines));
        assertEquals(150, Units.totalHp(tanks));
        assertEquals(190, Units.totalHp(mixed));
    }
}
```

세 단언 모두 통과한다. 메서드 하나가 세 종류의 목록을 받는다.

읽기 전용 목록에 넣으려 하면 컴파일 에러

`totalHp` 안에서 마린 한 기를 넣어 보면?

```
static int totalHp(Collection<? extends Unit> units) {
    units.add(Marine.create());
    // 컴파일 에러: incompatible types: Marine cannot be converted to CAP#1
    //   CAP#1 extends Unit from capture of ? extends Unit
    ...
}
```

- `units`의 실체는 `List<SiegeTank>`일 수도 있다. 컴파일러는 어느 쪽인지 모르니 아무것도 넣지 못하게 막는다 (`null`만 예외).
- `? extends`는 **꺼내 쓰는** 매개변수에 붙인다. 넣는 쪽은 다른 도구가 필요하다.

이번엔 넣는 쪽: addMarine은 List<Unit>에 못 넣는다

새로 뽑은 마린을 목록에 넣는 유틸이다. `Units.java`에 이 메서드를 추가한다.

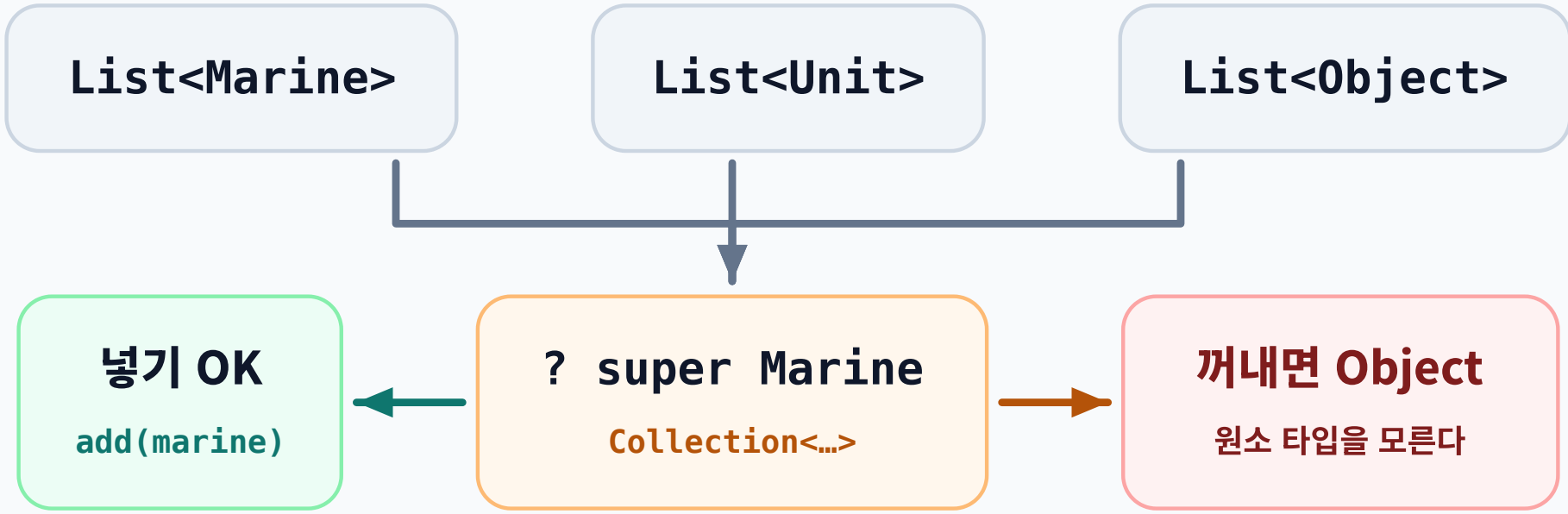
```
static void addMarine(Collection<Marine> destination, Marine marine) {
    destination.add(marine);
}
```

섞어 담는 부대 목록에 넣으려 하면 또 막힌다.

```
List<Unit> units = new ArrayList<>();
Units.addMarine(units, Marine.create());
// 컴파일 에러: incompatible types: List<Unit> cannot be converted to Collection<Marine>
```

마린은 `List<Unit>`에도, `List<Object>`에도 들어갈 수 있다. 그런데 `? extends Marine`으로 바꾸면 넣지 못한다. 필요한 건 반대 방향이다.

'Marine을 받아 줄 수 있는' 목록



`? super Marine`은 "Marine이거나 그 상위 타입의 목록"이다. 마린을 넣는 것은 언제나 안전하고, 꺼내면 `Object`로만 안다. 그래서 **쓰기 전용**이다.

addMarine(Collection<? super Marine>, Marine) 구현과 테스트

`Units.java`의 `addMarine()`을 이것으로 바꾼다.

```
// 넣기만 한다 → ? super Marine
static void addMarine(Collection<? super Marine> destination, Marine marine) {
    destination.add(marine);
}
```

`UnitsTest.java`에 이 테스트 메서드를 추가한다.

```
@Test
@DisplayName("addMarine은 List<Marine>, List<Unit>, List<Object> 모두에 마린을 넣는다")
void addMarineAcceptsSupertypeLists() {
    List<Marine> marines = new ArrayList<>();
    List<Unit> units = new ArrayList<>();
    List<Object> objects = new ArrayList<>();
    Marine marine = Marine.create();

    Units.addMarine(marines, marine);
    Units.addMarine(units, marine);
    Units.addMarine(objects, marine);

    assertSame(marine, marines.get(0));
    assertSame(marine, units.get(0));
    assertSame(marine, objects.get(0));
}
```

고르는 규칙: 읽기 전용은 extends, 쓰기 전용은 super



- 메서드가 매개변수에서 꺼내 **읽기만** 한다 → `? extends T` (읽기 전용)
- 메서드가 매개변수에 **넣기만** 한다 → `? super T` (쓰기 전용)
- 꺼내기도 넣기도 한다 → `T` (와일드카드 없음, 둘 다 가능)

transferTo의 매개변수 타입은? (퀴즈)

퀴즈

부대의 유닛을 모두 다른 목록(`List<Unit>`일 수도, 보관용 `List<Object>`일 수도)으로 옮기고 부대를 비운다.

```
class Squad {
    private final List<Unit> units = new ArrayList<>();

    void transferTo(/* ??? */ destination) {
        destination.addAll(units);
        units.clear();
    }
}
```

1. `Collection<Unit>`
2. `Collection<? extends Unit>`
3. `Collection<? super Unit>`
4. `Collection<?>`

풀이: Collection<? super Unit> destination

정답

정답은 3번이다. `destination`에는 유닛을 **넣기만** 한다. 쓰기 전용 매개변수다.

보기	결과
1	컴파일은 되지만 `List<Object>` 보관함을 넘기지 못한다
2	`addAll` 줄이 컴파일 에러: 무엇의 목록인지 몰라 넣을 수 없다
3	`List<Unit>`, `List<Object>` 모두 받는다
4	2와 같다. 원소 타입을 모르니 넣을 수 없다

다음 장에서 3번 선언을 `Squad`에 붙여 확인한다.

풀이 확인: transferTo로 List<Object> 보관함에 옮긴다

`Squad.java`에 이 메서드를 붙이고, 맨 위에 `import java.util.Collection;`을 추가한다.

```
void transferTo(Collection<? super Unit> destination) {
    destination.addAll(units);
    units.clear();
}
```

테스트를 실행하면 「보관함 2, 부대 0」이 통과한다. 확인이 끝나면 `transferTo`와 테스트 파일은 지운다(실습에서 빠진다).

> `List<Object>`는 동작을 보이려고 만든 예다. `Unit` 위 타입은 `Object` 뿐이라 `? super Unit`은 실무에서 드물다. 실제로는 `addMarine`처럼 **마린을 `List<Unit>`에 넣을 때** 쓴다.

SquadTransferTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadTransferTest {

    @Test
    @DisplayName("부대 유닛 2기를 List<Object> 보관함으로 옮기면 보관함 2, 부대 0")
    void transferToObjectList() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(SiegeTank.create());
        List<Object> archive = new ArrayList<>();

        squad.transferTo(archive);

        assertEquals(2, archive.size());
        assertEquals(0, squad.size());
    }
}
```

JDK도 이렇게 쓴다: Collections.addAll, Collections.sort

```
static <T> boolean addAll(Collection<? super T> c, T... elements)  // 받아 주는 쪽: super
static <T> void sort(List<T> list, Comparator<? super T> c)        // 비교기는 T를 받아 쓴다: super
```

Day 1 8교시 `Collections.sort(books, Comparator.comparing(Book::getTitle))`의 비교기 자리가 바로 이 선언이다. 호출해 보면(발췌):

```
List<Unit> units = new ArrayList<>();
Collections.addAll(units, Marine.create(), Marine.create()); // T = Marine

Comparator<Unit> byHp = new Comparator<Unit>() {
    @Override
    public int compare(Unit a, Unit b) {
        return Integer.compare(a.getHp(), b.getHp());
    }
};
List<Marine> marines = new ArrayList<>();
marines.add(Marine.ofDamaged(30));
marines.add(Marine.ofDamaged(10));
marines.add(Marine.create());

Collections.sort(marines, byHp); // Unit 비교기로 마린 목록을 정렬: 10, 30, 40
```

익명 클래스는 4교시에 람다 한 줄로 줄인다.

Squad.addAll이 마린 목록도 받게 한다 (실습)

실습 1교시 `Squad.java` 전체를 이것으로 바꾼다. 시작 코드의 `addAll`은 `Collection<Unit>`만 받는다.

Squad.java

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;

class Squad {
    private final List<Unit> units = new ArrayList<>();

    void add(Unit unit) { units.add(unit); }
    void addAll(Collection<Unit> newUnits) { units.addAll(newUnits); }
    List<Unit> getUnits() { return new ArrayList<>(units); }
    int size() { return units.size(); }

    int totalHp() {
        int total = 0;
        for (Unit unit : units) {
            total += unit.getHp(); // 40 + 40 + 60 = 140
        }
        return total;
    }
}
```

실습: SquadTest로 막힌 곳을 보고, 매개변수를 고친다

실습

`SquadTest.java` 전체를 바꾼다

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadTest {

    @Test
    @DisplayName("마린 2 + SCV 1 부대의 체력 합은 140")
    void totalHpOfMixedSquad() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(Marine.create());
        squad.add(SCV.create());

        int total = squad.totalHp();

        assertEquals(140, total);
    }
```

`SquadTest.java` (이어서)

```java
    @Test
    @DisplayName("마린 목록을 한 번에 추가하면 부대 크기 3")
    void addAllMarines() {
        Squad squad = new Squad();
        List<Marine> marines = new ArrayList<>();
        marines.add(Marine.create());
        marines.add(Marine.create());
        marines.add(Marine.create());

        // squad.addAll(marines); // 실습: 주석을 풀고 컴파일 에러를 고친다

        assertEquals(3, squad.size());
    }
}
```

1. 실행: 「부대 크기 3」이 `expected: <3> but was: <0>`
2. 주석을 풀면 `List<Marine> cannot be converted to Collection<Unit>`
3. `Squad.addAll`의 매개변수를 고쳐 통과시킨다

실습 풀이: newUnits는 꺼내 주는 쪽이다

정답

`Squad.java`의 `addAll()`을 이것으로 바꾼다.

```
void addAll(Collection<? extends Unit> newUnits) {
    units.addAll(newUnits);
}
```

- `newUnits`에서 유닛을 **꺼내기만** 하니 읽기 전용이다 → `? extends Unit`.
- 「부대 크기 3」과 1교시의 「체력 합 140」이 함께 통과한다.
- JDK의 `Collection.addAll(Collection<? extends E> c)`와 같은 선언이다.

정리: 와일드카드는 매개변수에, 반환 타입에는 쓰지 않는다

매개변수의 역할	선언	이번 시간의 예
읽기 전용: 꺼내기만 한다	`Collection<? extends T>`	`Units.totalHp`, `Squad.addAll`
쓰기 전용: 넣기만 한다	`Collection<? super T>`	`Units.addMarine`, `Squad.transferTo`
둘 다: 꺼내기도 넣기도 한다	`Collection<T>` (와일드카드 없음)	—
원소 타입과 상관없다	`Collection<?>`	`size()`만 쓰는 메서드
반환 타입	와일드카드 없이 `List<Unit>`	`Squad.getUnits()`

> 반환 타입에 `List<? extends Unit>`을 쓰면 호출하는 쪽이 와일드카드를 떠안는다. 받은 목록에 아무것도 넣지 못한다. 와일드카드는 사용자가 의식하지 않아도 "그냥 넘기면 되는" 자리에만 둔다. 아이템 31.

3교시

경고와 캐스팅을 없애는 두 도구

2교시에는 매개변수를 넓혔다. 이번에는 메서드 자체와 보관함에 타입을 싣는다.

두 부대의 유닛 집합을 합친다: Raw 타입 union

알파·브라보 부대의 마린을 한 집합으로 합치는 유틸이다. SetUtils.java

```java
import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

/** before: Raw 타입 집합 유틸. 컴파일러가 원소 타입을 모른다. */
final class SetUtils {
    private SetUtils() {
    }

    static Set union(Set s1, Set s2) {
        Set result = new HashSet(s1);
        result.addAll(s2);
        return Collections.unmodifiableSet(result);
    }
}
```

원소 타입이 없으니 호출부는 `Object`로 받아 캐스팅한다(발췌, 마린 집합 `alpha`·`bravo`를 합칠 때).

```java
Set merged = SetUtils.union(alpha, bravo);
for (Object o : merged) {
    Marine marine = (Marine) o;
    marine.stimpack();
}
```

컴파일러 경고, 그리고 호출부마다 캐스팅

컴파일하면 경고부터 나온다.

```
$ javac SetUtils.java
Note: SetUtils.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.

$ javac -Xlint:unchecked SetUtils.java
SetUtils.java:11: warning: [unchecked] unchecked call to HashSet(Collection<? extends E>)
                  as a member of the raw type HashSet
SetUtils.java:12: warning: [unchecked] unchecked call to addAll(Collection<? extends E>)
                  as a member of the raw type Set
SetUtils.java:13: warning: [unchecked] unchecked method invocation: method unmodifiableSet ...
3 warnings
```

- 경고가 쌓이면 진짜 위험한 경고가 그 속에 묻힌다. 빌드 로그의 경고를 아무도 읽지 않게 된다.
- 반환이 Raw 타입 `Set`이라 호출하는 쪽은 `for (Object o : merged)` → `(Marine) o`를 매번 쓴다.
- 경고 메시지가 말하는 것: "원소 타입을 모르니 안전한지 확인할 수 없다."

탱크 집합을 섞어 넘겨도 컴파일된다

브라보 자리에 실수로 탱크 집합을 넘겼다. SetUtilsTest.java

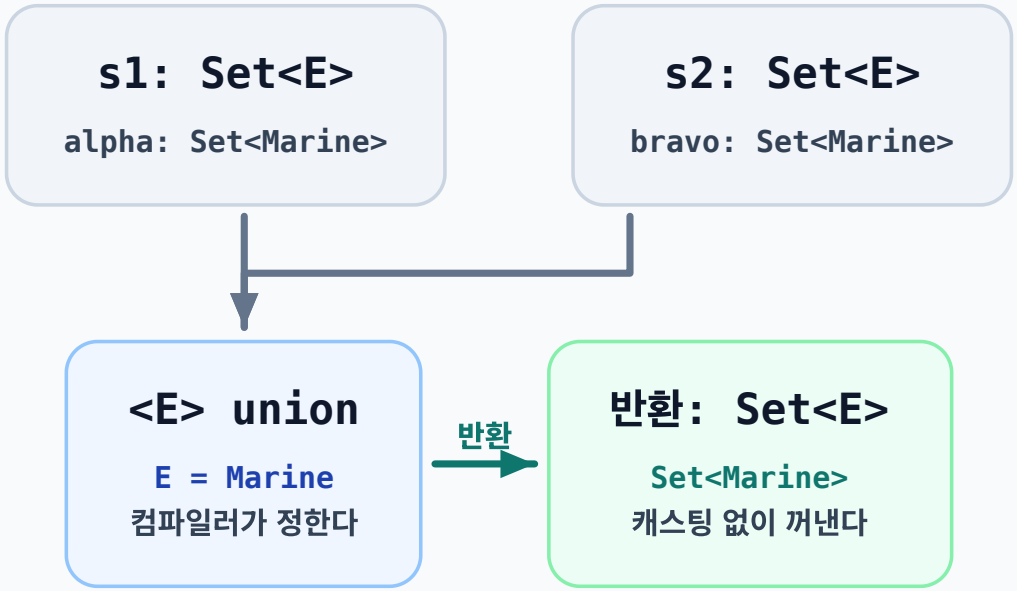
```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SetUtilsTest {
    @Test
    @DisplayName("합친 집합의 마린 모두에게 스팀팩을 쓰면 체력 합은 30이다")
    void unionWithTanksByMistake() {
        Set<Marine> alpha = new HashSet<>();
        alpha.add(Marine.create());
        Set<SiegeTank> bravo = new HashSet<>();
        bravo.add(SiegeTank.create());

        Set merged = SetUtils.union(alpha, bravo); // 탱크 집합을 넘겨도 컴파일된다
        int totalHp = 0;
        for (Object o : merged) {
            Marine marine = (Marine) o;
            marine.stimpack();
            totalHp += marine.getHp();
        }
        assertEquals(30, totalHp);
    }
}
```

실행: `ClassCastException: class SiegeTank cannot be cast to class Marine`. 실수는 `union` 줄에서 했는데 터진 곳은 꺼내는 줄이다. 메서드에 원소 타입을 적을 수는 없을까?

타입 매개변수를 메서드에 선언한다



- 클래스가 아니라 메서드에 `<E>`를 단다. `s1`, `s2`, 반환 타입이 같은 `E`로 묶인다.
- 부를 때 넘긴 인자를 보고 컴파일러가 `E`를 정한다. 이것을 **제네릭 메서드**라 한다.
- 이펙티브 자바 아이템 30: 이왕이면 제네릭 메서드로 만들라.

union 제네릭 메서드 구현

`SetUtils.java`를 이것으로 바꾼다. SetUtils.java

```
import java.util.Collections;
import java.util.HashSet;
import java.util.Set;

final class SetUtils {
    private SetUtils() {
    }

    static <E> Set<E> union(Set<E> s1, Set<E> s2) {
        Set<E> result = new HashSet<>(s1);
        result.addAll(s2);
        return Collections.unmodifiableSet(result);
    }
}
```

- 타입 매개변수 목록 `<E>`는 `static`과 반환 타입 사이에 쓴다.
- 몸통은 그대로이고, `Set` 네 군데에 `<E>`가 붙었다. `-Xlint:unchecked` 경고는 0개다.

테스트: Set<Marine> 두 개를 합치면 Set<Marine>

`SetUtilsTest.java` 전체를 이것으로 바꾼다. SetUtilsTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SetUtilsTest {
    @Test
    @DisplayName("마린 집합 두 개를 합치면 캐스팅 없이 Set<Marine>을 받는다")
    void unionMarinesWithoutCast() {
        Set<Marine> alpha = new HashSet<>();
        alpha.add(Marine.create());
        alpha.add(Marine.create());
        Set<Marine> bravo = new HashSet<>();
        bravo.add(Marine.create());

        Set<Marine> merged = SetUtils.union(alpha, bravo);
        int totalHp = 0;
        for (Marine marine : merged) {
            marine.stimpack();
            totalHp += marine.getHp();
        }
        assertEquals(3, merged.size());
        assertEquals(90, totalHp);
    }
}
```

통과한다. `(Marine) o` 캐스팅이 사라졌다. `SetUtils.<Marine>union(...)`으로 적지 않아도 컴파일러가 인자를 보고 `E = Marine`을 고른다(**타입 추론**). `Collections.emptyList()`에 `<String>`을 안 적던 이유다.

union(마린 집합, 탱크 집합)은 컴파일될까? (퀴즈)

퀴즈

방금 만든 `static <E> Set<E> union(Set<E> s1, Set<E> s2)`로 마린과 탱크를 한 `Set<Unit>`에 합치려 한다.

```
Set<Marine> marines = new HashSet<>();
Set<SiegeTank> tanks = new HashSet<>();

Set<Unit> all = SetUtils.union(marines, tanks);
```

1. 컴파일된다. `E`가 `Unit`으로 정해진다.
2. 컴파일된다. `E`가 `Object`로 정해진다.
3. 컴파일 에러. 하나의 `E`로 두 인자를 동시에 만족시킬 수 없다.
4. 컴파일되지만 실행 중 `ClassCastException`이 난다.

풀이: 안 된다 → Set<? extends E>로 넓힌다

정답

정답은 3번이다. 마린 집합은 유닛 집합이 아니어서(2교시), `E`가 `Marine`이면서 동시에 `SiegeTank`여야 한다.

```
found:  Set<Marine>,Set<SiegeTank>
reason: inference variable E has incompatible equality constraints SiegeTank,Marine
```

`s1`, `s2`에서는 꺼내기만 한다(읽기 전용) → 2교시 규칙대로 `? extends E`. 앞에서 만든 `SetUtils.java`의 매개변수를 넓힌다: `union`을 이것으로 바꾼다. SetUtils.java

```java
static <E> Set<E> union(Set<? extends E> s1, Set<? extends E> s2) {
    Set<E> result = new HashSet<>(s1);
    result.addAll(s2);
    return Collections.unmodifiableSet(result);
}
```

이제 `SetUtils.union(marines, tanks)`는 `E = Unit`으로 컴파일된다. 다음 장의 테스트로 확인한다.

테스트: 마린과 탱크를 Set<Unit>으로 받는다

새 테스트 파일을 만든다. 퀴즈와 달리 집합마다 유닛을 한 기씩 넣는다. SetUtilsWildcardTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.HashSet;
import java.util.Set;
import static org.junit.jupiter.api.Assertions.*;

class SetUtilsWildcardTest {
    @Test
    @DisplayName("마린 집합과 탱크 집합을 합쳐 Set<Unit>으로 받는다")
    void unionMarinesAndTanksAsUnits() {
        Set<Marine> marines = new HashSet<>();
        marines.add(Marine.create());
        Set<SiegeTank> tanks = new HashSet<>();
        tanks.add(SiegeTank.create());

        Set<Unit> all = SetUtils.union(marines, tanks); // E = Unit

        assertEquals(2, all.size());
        assertEquals(190, Units.totalHp(all)); // 40 + 150
    }
}
```

`Units.totalHp`는 2교시에 만든 `Units.java`의 메서드다. `SetUtilsTest`와 함께 2건 통과.

새 요구: 유닛마다 타입이 다른 메타데이터를 붙인다

부대 화면이 유닛 옆에 여러 정보를 함께 보여 달라고 한다.

정보	값의 타입	예
집결지	`Position`	`new Position(10, 20)`
마지막 체력	`Health`	`new Health(30, 40)`
별명	`String`	`"돌격대장"`
처치 수	`Integer`	`3`

- 값마다 타입이 다르다. 칸이 늘어날 수도 있어서 필드를 하나씩 만들기도 부담스럽다.
- 한 보관함에 서로 다른 타입을 담고, 꺼낼 때는 각자의 타입으로 받고 싶다.

Map<String, Object>에 담으면

현업의 모델·파라미터 맵처럼 문자열 키에 `Object` 값을 담고, 꺼낼 때마다 캐스팅한다. MapMetadataTest.java

```java
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import java.util.HashMap;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.*;

class MapMetadataTest {
    private final Map<String, Object> metadata = new HashMap<>();

    @BeforeEach
    void setUp() {
        metadata.put("rallyPoint", new Position(10, 20));
        metadata.put("lastHealth", new Health(30, 40));
        metadata.put("nickname", "돌격대장");
        metadata.put("kills", "3"); // 다른 화면이 요청 파라미터 문자열을 그대로 넣었다
    }

    @Test
    @DisplayName("꺼낼 때마다 캐스팅하면 집결지와 체력을 읽을 수 있다")
    void readWithCasts() {
        Position rallyPoint = (Position) metadata.get("rallyPoint");
        Health lastHealth = (Health) metadata.get("lastHealth");
        assertEquals(10, rallyPoint.getX());
        assertEquals(30, lastHealth.getHp());
    }
}
```

키 오타와 잘못된 캐스팅은 꺼낼 때 터진다

같은 맵에서 두 가지를 더 꺼내 본다. `MapMetadataTest.java`에 두 테스트를 더한다. MapMetadataTest.java

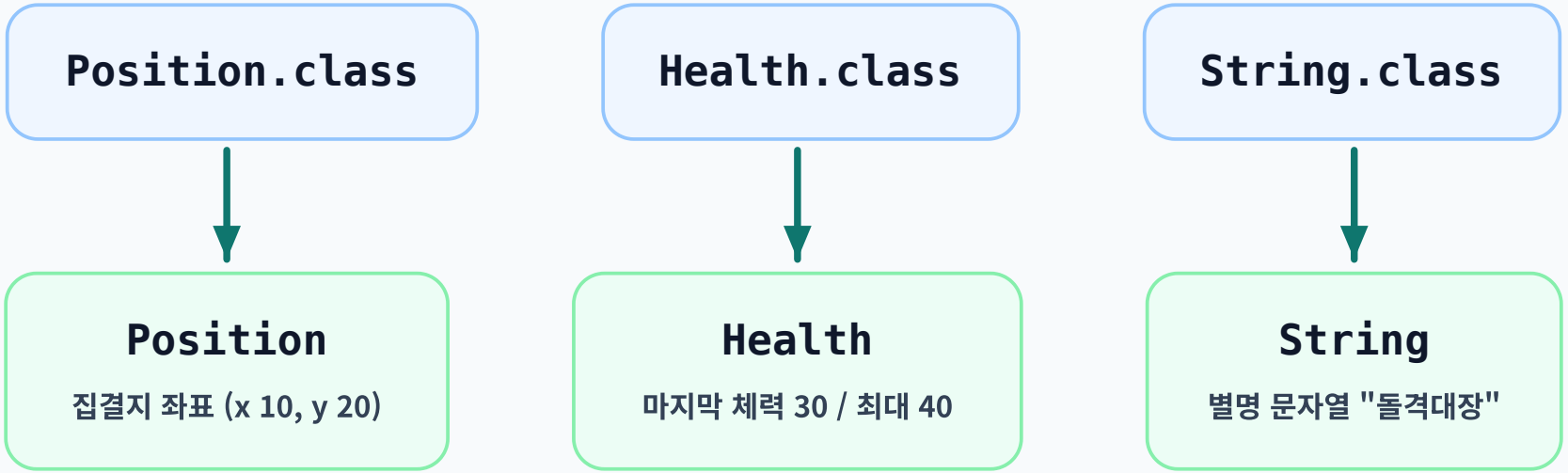
```
@Test
@DisplayName("집결지의 x 좌표는 10이다")
void rallyPointWithTypoKey() {
    Position rallyPoint = (Position) metadata.get("rallypoint"); // 키 오타

    assertEquals(10, rallyPoint.getX());
    // 실패: NullPointerException — 없는 키라 null을 받았다
}

@Test
@DisplayName("처치 수는 3이다")
void killsStoredAsString() {
    Integer kills = (Integer) metadata.get("kills");
    // 실패: ClassCastException: class java.lang.String cannot be cast to class java.lang.Integer
    assertEquals(3, kills.intValue());
}
```

둘 다 컴파일은 통과했다. 키와 값의 타입이 따로 놀기 때문이다. 키가 값의 타입을 알고 있다면?

타입 자체를 키로 쓴다



- 문자열 키 대신 `Position.class`, `Health.class` 같은 **타입**을 키로 쓴다. 키가 곧 값의 타입이라 오타도, 엉뚱한 캐스팅도 없다.
- 타입마다 다른 값을 담되 꺼낼 때 타입이 보장되는 보관함을 **타입 안전 이종 컨테이너**라 한다.
- 이펙티브 자바 아이템 33: 타입 안전 이종 컨테이너를 고려하라.

Position.class의 타입은 Class<Position>: 타입 토큰

새 테스트 파일을 만든다. `UnitMetadataContainerTest.java`

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class UnitMetadataContainerTest {
    @Test
    @DisplayName("타입 토큰은 그 타입의 Class 객체다")
    void classLiteralIsTypeToken() {
        Class<Position> type = Position.class;

        assertEquals("Position", type.getSimpleName()); // 통과
    }
}
```

- `Class` 자체가 제네릭 클래스다. `Position.class`는 `Class<Position>`이다.
- 그래서 `Class<T>`를 키로 받으면 그 `T`로 "값의 타입"까지 함께 전달된다. 이렇게 타입 정보를 실어 나르는 `Class` 객체를 **타입 토큰**이라 한다.
- `type.cast(obj)`는 실행 중에 "obj가 정말 `T`인가?"를 검사하고 `T`로 돌려준다. 아니면 `ClassCastException`이다.

UnitMetadataContainer 최소 구현: type.cast로 넣고 꺼낸다

UnitMetadataContainer.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.Objects;

final class UnitMetadataContainer {
    private final Map<Class<?>, Object> values = new HashMap<>();

    <T> void putMetadata(Class<T> type, T instance) {
        Objects.requireNonNull(type, "type");
        values.put(type, type.cast(instance));
    }

    <T> T getMetadata(Class<T> type) {
        return type.cast(values.get(type));
    }
}
```

- 맵 자체는 키와 값 타입을 엮지 않는다. 그 연결은 두 메서드의 `<T>`가 지킨다.
- `getMetadata` : `type.cast`가 `Object`를 `T`로 바꾼다. `(T)` 캐스팅도, 경고도 없다.
- `putMetadata` : Raw 타입 `Class`로 속여 넣어도 `type.cast`가 넣는 순간 `ClassCastException`을 던진다.

테스트: 캐스팅 없이 꺼낸다

`UnitMetadataContainerTest.java`에 테스트를 더한다. UnitMetadataContainerTest.java

```
@Test
@DisplayName("타입을 키로 넣으면 캐스팅 없이 그 타입으로 꺼낸다")
void putAndGetWithoutCast() {
    UnitMetadataContainer metadata = new UnitMetadataContainer();
    metadata.putMetadata(Position.class, new Position(10, 20));
    metadata.putMetadata(Integer.class, 3);

    Position rallyPoint = metadata.getMetadata(Position.class);
    int kills = metadata.getMetadata(Integer.class);

    assertEquals(new Position(10, 20), rallyPoint);
    assertEquals(3, kills);
}
```

```
metadata.putMetadata(Integer.class, "3");
// 컴파일 에러: inference variable T has incompatible bounds
//   equality constraints: Integer, lower bounds: String
```

처치 수에 문자열을 넣던 실수가 이제는 **컴파일 때** 막힌다.

containsMetadata 추가하기 (실습)

실습

`UnitMetadataContainer.java`에 예외만 던지는 메서드를 더한다. UnitMetadataContainer.java

```
boolean containsMetadata(Class<?> type) {
    throw new UnsupportedOperationException(
            "실습: containsMetadata");
}
```

1. `UnitMetadataContainerTest` 실행: `UnsupportedOperationException: 실습: containsMetadata`로 실패한다.
2. 메서드 몸통을 한 줄로 채운다. 값을 꺼낼 필요가 없으니 `<T>`도 필요 없다.
3. 오른쪽 테스트가 통과하면 끝.

`UnitMetadataContainerTest.java`에 테스트를 더한다. UnitMetadataContainerTest.java

```
@Test
@DisplayName("Position을 넣으면 true, Health는 넣지 않았으면 false")
void containsOnlyPutTypes() {
    UnitMetadataContainer metadata = new UnitMetadataContainer();
    metadata.putMetadata(Position.class, new Position(10, 20));

    assertTrue(metadata.containsMetadata(Position.class));
    assertFalse(metadata.containsMetadata(Health.class));
}
```

실습 풀이: 키가 있는지만 본다

정답

`UnitMetadataContainer.java`의 `containsMetadata`를 이것으로 바꾼다. UnitMetadataContainer.java

```
boolean containsMetadata(Class<?> type) {
    return values.containsKey(type);
}
```

- 값을 돌려주지 않으니 타입을 묶을 `<T>`가 필요 없다. 어떤 타입 토큰이든 받는 `Class<?>`면 된다.
- 2교시 정리의 「원소 타입과 상관없다 → `<?>`」 자리다.
- `getMetadata(...) != null`로 확인하면 `null` 값을 넣은 경우와 구별되지 않는다. 키를 직접 본다.

정리: 메서드에 <T>, 키에 Class<T>

불편	도구	이번 시간의 예
Raw 타입 유틸의 경고, 호출부 캐스팅	제네릭 메서드 `<E>`	`SetUtils.union`, `SetUtils.intersection`
하위 타입 집합을 섞어 받기	입력에 `? extends E`	`union(Set<? extends E>, ...)`
호출부의 타입 인자	타입 추론	`SetUtils.union(alpha, bravo)`
타입마다 다른 값, 꺼낼 때 캐스팅	타입 안전 이종 컨테이너	`UnitMetadataContainer`
키와 값 타입 연결	타입 토큰 `Class<T>` + `type.cast`	`putMetadata(Position.class, ...)`
`List<String>`처럼 `<>`가 붙은 타입	키로 쓸 수 없다: `List<String>.class`는 컴파일 에러, `List.class`는 모든 `List`가 한 칸	`Position`, `Health`, `String`처럼 `<>` 없는 타입만 키로

> 아이템 30: 이왕이면 제네릭 메서드로 만들라. 아이템 33: 타입 안전 이종 컨테이너를 고려하라.

4교시

조건 하나에 여섯 줄

「살아 있는 공중 유닛만 쏜다」를 말하는 데 몇 줄이 필요할까?

마린의 공격 조건을 바꿔 끼운다: AttackStrategy

누구를 쏠지는 상황마다 다르다. 조건을 객체로 받아서 바꿔 끼운다.

AttackStrategy.java

```
interface AttackStrategy {
    boolean canAttack(Unit target);
}
```

앞에서 만든 Marine.java 에 `attack` 메서드를 더한다.

```
boolean attack(Unit target, AttackStrategy strategy) {
    if (strategy.canAttack(target)) {
        // 피해 계산은 이 세션의 관심사가 아니므로 생략한다
        return true;
    }
    return false;
}
```

`true` 면 공격했다는 뜻이다. `Unit` , `Wraith` 는 1교시의 Day 2 유닛 세트 그대로다.

공중 유닛만 노리는 조건을 익명 클래스로 쓰면

MarineAttackTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertTrue;

class MarineAttackTest {

    @Test
    @DisplayName("살아 있는 공중 유닛이면 공격한다 (익명 클래스)")
    void attackAliveAirUnitWithAnonymousClass() {
        Marine marine = Marine.create();
        Wraith wraith = Wraith.create();

        boolean attacked = marine.attack(wraith, new AttackStrategy() {
            @Override
            public boolean canAttack(Unit target) {
                return target.getHp() > 0 && target.isAirUnit(); // 하고 싶은 말
            }
        });

        assertTrue(attacked);
    }
}
```

테스트는 통과한다. 문제는 **읽기**다. 하고 싶은 말은 한 줄인데 조건을 넘기려고 여섯 줄을 쓴다.

추상 메서드가 하나인 인터페이스는 람다로 쓴다

지운다: AttackStrategy 자리가 이미 알려 준다　　남는다: 매개변수와 조건식

```
new AttackStrategy() {
    @Override
    public boolean canAttack(Unit target) {
        return target.getHp() > 0 && target.isAirUnit();
    }
}
```

↓ 람다 한 줄

```
target -> target.getHp() > 0 && target.isAirUnit()
```

`attack`의 두 번째 자리가 `AttackStrategy`이고 메서드가 `canAttack` 하나뿐이니, 컴파일러는 나머지를 안다. 이렇게 쓰는 것이 **람다**다. 이펙티브 자바 아이템 42: 익명 클래스보다는 람다를 사용하라.

> 현업 장면: 콜백이 필요할 때마다 새 인터페이스를 만들고 익명 클래스로 채우던 레거시 코드가 딱 이 모양이다.

람다로 바꾸고 테스트로 확인

`Marine.attack`은 그대로다. 넘기는 쪽만 바뀐다. 1일차 `assertThrows`에 `() -> ...`로 넘기던 것이 바로 이 문법이다.

앞에서 만든 MarineAttackTest.java의 테스트 메서드를 이것으로 바꾼다.

```
@Test
@DisplayName("살아 있는 공중 유닛이면 공격한다 (람다)")
void attackAliveAirUnitWithLambda() {
    Marine marine = Marine.create();
    Wraith wraith = Wraith.create();

    boolean attacked = marine.attack(wraith,
            target -> target.getHp() > 0 && target.isAirUnit());

    assertTrue(attacked);
}
```

```
✓ 살아 있는 공중 유닛이면 공격한다 (람다)
```

람다를 더 짧게: 타입, 괄호, return 생략

네 줄 모두 같은 조건이다.

생략한 것	람다
없음	`(Unit target) -> { return target.isAirUnit(); }`
매개변수 타입	`(target) -> { return target.isAirUnit(); }`
매개변수 괄호	`target -> { return target.isAirUnit(); }`
중괄호와 `return`	`target -> target.isAirUnit()`

- 타입은 자리(`AttackStrategy`)가 알려 주니 쓰지 않는 것이 기본이다.
- 괄호는 매개변수가 하나일 때만 생략한다. 둘이면 `(a, b) -> a + b` .
- 중괄호와 `return` 은 함께 뺀다. 하나만 빼면 컴파일 에러다.

메서드 참조 맛보기: Unit::getName, Marine::create

람다	메서드 참조	가리키는 것
`target -> target.isAirUnit()`	`Unit::isAirUnit`	받은 객체의 메서드
`u -> u.getName()`	`Unit::getName`	받은 객체의 메서드
`() -> Marine.create()`	`Marine::create`	정적 메서드

`MarineAttackTest.java`에 테스트 메서드를 하나 더한다.

```
@Test
@DisplayName("메서드 참조 Unit::isAirUnit도 같은 조건으로 쓸 수 있다")
void attackWithMethodReference() {
    Marine marine = Marine.create();

    assertTrue(marine.attack(Wraith.create(), Unit::isAirUnit));
}
```

> 람다가 메서드 하나를 그대로 부르기만 하면 메서드 이름으로 줄일 수 있다. 아이템 43(람다보다는 메서드 참조를 사용하라)은 이름만 알아 둔다.

익명 클래스를 남겨야 하는 경우는? (퀴즈)

퀴즈

람다로 바꾸지 말고 익명 클래스로 남겨야 하는 경우를 **모두** 고르세요.

1. 구현할 인터페이스에 추상 메서드가 두 개 있다
2. 조건식이 한 줄로 끝난다
3. 구현 안에서 `this`로 자기 자신(그 전략 객체)을 가리켜야 한다
4. 매개변수 타입을 적고 싶다

풀이: 추상 메서드가 여럿이거나, 자기 자신(this)을 가리켜야 할 때

정답

정답은 1번과 3번이다.

경우	이유
추상 메서드가 둘 이상	람다는 메서드 하나의 몸통만 쓸 수 있다
`this`로 자기 자신을 가리킴	람다 안의 `this`는 람다를 감싼 바깥 객체다
(덤) 추상 클래스를 구현	람다는 인터페이스 자리에만 들어간다

2번은 람다가 가장 빛나는 경우이고, 4번은 `(Unit target) -> ...`처럼 람다에서도 적을 수 있다.

> 람다는 몇 줄을 넘기면 이름이 없는 만큼 읽기 어려워진다. 길어지면 이름 있는 메서드로 뺀다.

조건이 늘어난다: TargetFilter, HpRule, UnitNamer… 인터페이스가 쌓인다

기능마다, 담당자마다 비슷한 인터페이스를 하나씩 만들었다.

발췌: 실무에서 흔히 보는 모양(앞에서 만든 `AttackStrategy`는 그대로 둔다)

```
interface AttackStrategy {
    boolean canAttack(Unit target);   // Unit -> boolean
}

interface HpRule {
    boolean check(Unit unit);         // Unit -> boolean
}

interface UnitNamer {
    String name(Unit unit);           // Unit -> String
}
```

넷 중 셋이 모양이 똑같다(`Unit -> boolean`). 이름만 다르다.

새로 만드는 파일은 하나다. 다음 장 시연에 쓴다.

TargetFilter.java

```
interface TargetFilter {
    boolean test(Unit unit);          // Unit -> boolean
}
```

직접 만든 인터페이스는 조합이 안 된다: and, negate를 매번 직접

「살아 있고 공중」과 「공중이 아님」을 만들려면 손으로 이어 붙인다.

CustomCombineTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class CustomCombineTest {

    @Test
    @DisplayName("직접 만든 인터페이스는 and와 negate를 손으로 써야 한다")
    void combineByHand() {
        AttackStrategy alive = t -> t.getHp() > 0;
        AttackStrategy air = t -> t.isAirUnit();

        AttackStrategy aliveAir = t -> alive.canAttack(t) && air.canAttack(t);
        AttackStrategy ground = t -> !air.canAttack(t);

        Marine marine = Marine.create();
        assertTrue(marine.attack(Wraith.create(), aliveAir));
        assertFalse(marine.attack(Wraith.create(), ground));
    }
}
```

모양이 같아도 TargetFilter는 attack에 넘길 수 없다

`TargetFilter`도 `Unit -> boolean`이지만 `attack`의 자리는 `AttackStrategy`다.
앞에서 만든 `CustomCombineTest`의 `combineByHand()` 끝, 마지막 `assertFalse` 줄 아래에 두 줄을 붙여 본다.
`marine`은 그 메서드에서 이미 만든 변수다.

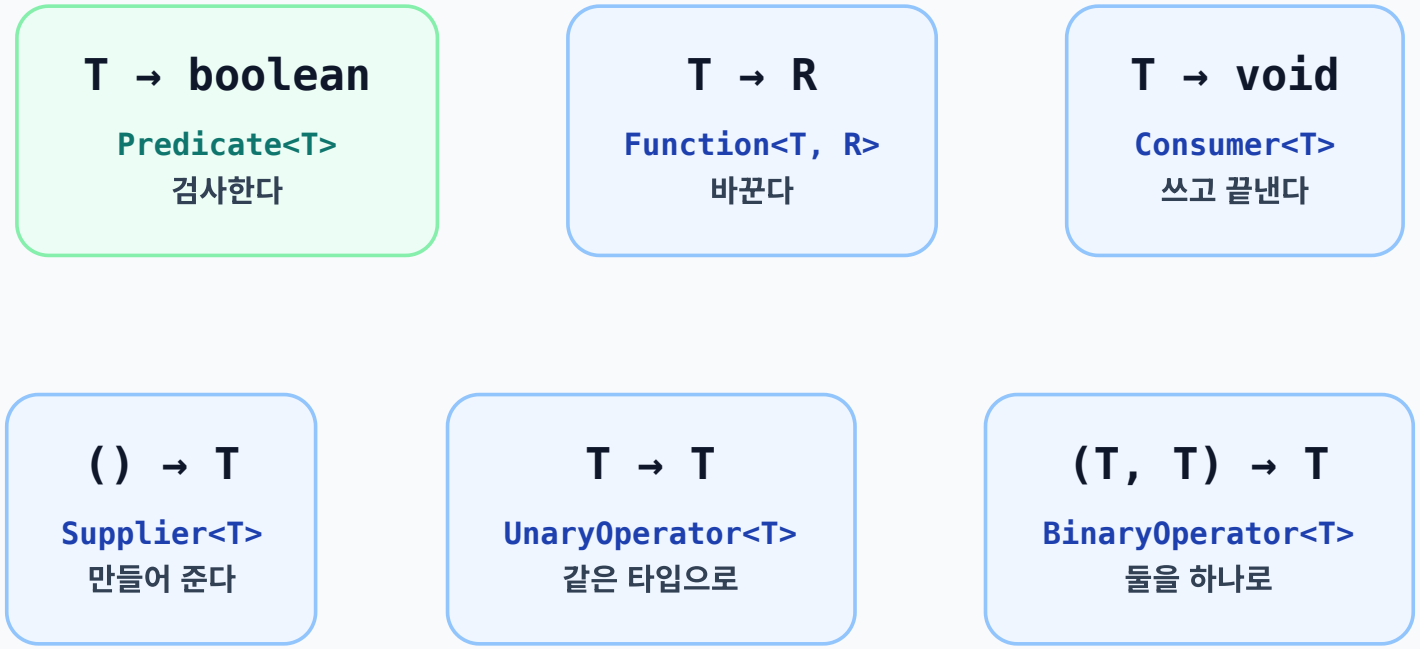
```
TargetFilter airFilter = t -> t.isAirUnit();
marine.attack(Wraith.create(), airFilter); // 컴파일 에러: incompatible types
```

```
error: incompatible types: TargetFilter cannot be converted to AttackStrategy
```

이 두 줄은 컴파일 에러를 보는 시연이다. 확인했으면 지운다.

모양이 같은 인터페이스를 한 번만, 조합 메서드까지 갖춰 만들어 두면 어떨까?

자바가 이미 만들어 둔 모양: java.util.function



입력과 출력 모양마다 하나씩, Java 8이 `java.util.function`에 이미 만들어 두었다.
이펙티브 자바 아이템 44: 표준 함수형 인터페이스를 사용하라.

기본 여섯 가지와 우리 코드의 예

인터페이스	메서드	우리 코드의 예
`Predicate<Unit>`	`test`	`u -> u.getHp() > 0`
`Function<Unit, String>`	`apply`	`Unit::getName`
`Consumer<String>`	`accept`	`System.out::println`
`Supplier<Marine>`	`get`	`Marine::create`
`UnaryOperator<Health>`	`apply`	`h -> h.restore(10)`
`BinaryOperator<Integer>`	`apply`	`Integer::sum`

`h -> h.restore(10)`이 새 `Health`를 돌려주는 것은 1일차에 `Health`를 불변으로 바꾼 덕분이다.

> 기본 타입 전용도 있다(`IntPredicate`, `IntBinaryOperator` 등). `int`를 `Integer`로 박싱하지 않으려면 그쪽을 쓴다.

AttackStrategy를 Predicate<Unit>으로 바꾼다

`Unit -> boolean` 모양이면 `Predicate<Unit>`이다. 앞에서 만든 Marine.java 의

`attack`을 이것으로 바꾸고, 파일 맨 위에 `import java.util.function.Predicate;`를 더한다.

```
boolean attack(Unit target, Predicate<Unit> condition) {
    if (condition.test(target)) {
        // 피해 계산은 이 세션의 관심사가 아니므로 생략한다
        return true;
    }
    return false;
}
```

`AttackStrategy.java`, `TargetFilter.java`, `CustomCombineTest.java`는 지운다.
`MarineAttackTest`의 람다는 한 글자도 바뀌지 않고 그대로 통과한다. 람다는 모양만 맞으면 어느 자리에든 들어간다.

```
boolean attacked = marine.attack(Wraith.create(),
        target -> target.getHp() > 0 && target.isAirUnit());
```

조건에 이름을 붙여 TargetRules에 모은다

자주 쓰는 조건은 `Predicate`를 돌려주는 정적 메서드로 이름을 붙인다.

TargetRules.java

```
import java.util.function.Predicate;

final class TargetRules {

    private TargetRules() {
    }

    static Predicate<Unit> isAir() {
        return u -> u.isAirUnit();
    }

    static Predicate<Unit> isAlive() {
        return u -> u.getHp() > 0;
    }
}
```

> `Predicate`에는 앞에서 손으로 쓰던 `and`, `or`, `negate`가 이미 있다.

Predicate.and로 조건을 조합한다

MarinePredicateAttackTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class MarinePredicateAttackTest {

    @Test
    @DisplayName("살아 있고 공중인 레이스는 공격한다")
    void attackAliveAirUnit() {
        Marine marine = Marine.create();

        boolean attacked = marine.attack(Wraith.create(),
                TargetRules.isAlive().and(TargetRules.isAir()));

        assertTrue(attacked);
    }

    @Test
    @DisplayName("체력 0인 레이스는 공중이어도 공격하지 않는다")
    void skipDeadAirUnit() {
        Marine marine = Marine.create();

        boolean attacked = marine.attack(Wraith.ofDamaged(0),
                TargetRules.isAlive().and(TargetRules.isAir()));

        assertFalse(attacked);
    }
```

negate로 뒤집고, 람다도 그대로 넘긴다

MarinePredicateAttackTest.java (이어서)

```
    @Test
    @DisplayName("negate로 지상 유닛만 노린다")
    void attackGroundUnitWithNegate() {
        Marine marine = Marine.create();

        assertTrue(marine.attack(Marine.create(), TargetRules.isAir().negate()));
        assertFalse(marine.attack(Wraith.create(), TargetRules.isAir().negate()));
    }

    @Test
    @DisplayName("AttackStrategy에 넘기던 람다를 그대로 넘길 수 있다")
    void sameLambdaStillWorks() {
        Marine marine = Marine.create();

        boolean attacked = marine.attack(Wraith.create(),
                target -> target.getHp() > 0 && target.isAirUnit());

        assertTrue(attacked);
    }
}
```

`isAlive().and(isAir())`가 「살아 있고 공중」이다. 반대는 `isAir().negate()`.

```
✓ 살아 있고 공중인 레이스는 공격한다
✓ 체력 0인 레이스는 공중이어도 공격하지 않는다
✓ negate로 지상 유닛만 노린다
✓ AttackStrategy에 넘기던 람다를 그대로 넘길 수 있다
```

시그니처를 보고 인터페이스를 고르세요 (퀴즈)

퀴즈

각 줄에 맞는 표준 함수형 인터페이스(타입 인자 포함)는?

번호	하는 일
1	유닛을 받아 이름(`String`)을 돌려준다
2	아무것도 받지 않고 새 `Marine`을 만들어 준다
3	`Health`를 받아 10 회복한 `Health`를 돌려준다
4	유닛을 받아 화면에 출력하고 끝난다
5	유닛을 받아 공격할지 `true` / `false`로 답한다
6	두 체력(정수)을 받아 합을 돌려준다

풀이: 입력과 출력 모양만 보면 된다

정답

번호	모양	인터페이스
1	`Unit → String`	`Function<Unit, String>`
2	`() → Marine`	`Supplier<Marine>`
3	`Health → Health`	`UnaryOperator<Health>`
4	`Unit → void`	`Consumer<Unit>`
5	`Unit → boolean`	`Predicate<Unit>`
6	`(Integer, Integer) → Integer`	`BinaryOperator<Integer>`

> 6번은 박싱을 피하려면 `IntBinaryOperator`가 낫다. 3번은 입력과 출력 타입이 같아서 `Function`보다 `UnaryOperator`다.

실습 준비: TargetRulesTest를 먼저 붙여 넣는다

실습 TargetRulesTest.java 다음 장 1단계에서 stub을 넣기 전까지는 빨간 줄이 정상이다.

```java
import java.util.function.Predicate;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class TargetRulesTest {

    @Test
    @DisplayName("체력 30인 레이스는 공격한다")
    void attackWoundedWraith() {
        Predicate<Unit> woundedAir = TargetRules.isAir().and(TargetRules.isWounded());

        boolean attacked = Marine.create().attack(Wraith.ofDamaged(30), woundedAir);

        assertTrue(attacked);
    }

    @Test
    @DisplayName("체력 20인 마린은 공중이 아니라서 공격하지 않는다")
    void skipWoundedMarine() {
        Predicate<Unit> woundedAir = TargetRules.isAir().and(TargetRules.isWounded());

        boolean attacked = Marine.create().attack(Marine.ofDamaged(20), woundedAir);

        assertFalse(attacked);
    }
}
```

isWounded 조건을 만들어 공중 조건과 조합한다 (실습)

실습

1. 앞에서 만든 `TargetRules.java` 에 상수와 시작 코드를 더한다.

```
static final int WOUNDED_HP = 50;   // 클래스 맨 위, 생성자 앞에

static Predicate<Unit> isWounded() {
    throw new UnsupportedOperationException("실습: isWounded");
}
```

2. `TargetRulesTest` 를 실행한다. 2건이 `UnsupportedOperationException: 실습: isWounded` 로 실패한다.
3. `isWounded()` 가 「체력이 `WOUNDED_HP` 보다 작다」는 `Predicate<Unit>` 을 돌려주게 고친다.
4. 다시 실행해 초록을 본다.

```
✓ 체력 30인 레이스는 공격한다
✓ 체력 20인 마린은 공중이 아니라서 공격하지 않는다
```

더 해 보기: `isAlive()` 도 붙여 체력 0인 레이스를 빼 보자.

실습 풀이: 조건 하나가 람다 한 줄이다

풀이

`TargetRules.java`의 `isWounded()`를 이것으로 바꾼다.

```
static Predicate<Unit> isWounded() {
    return u -> u.getHp() < WOUNDED_HP;
}
```

새 인터페이스 없이 조건을 하나 더 만들어 `and`로 바로 붙였다.
테스트의 `TargetRules.isAir().and(TargetRules.isWounded())`가 「공중이고 다쳤다」로 그대로 읽힌다.

정리: 직접 만들어도 되는 때

기본은 표준 인터페이스다. 아래에 해당할 때만 직접 만든다.

상황	예
자주 쓰이고, 이름 자체가 용도를 설명한다	`Comparator<T>` (모양은 `ToIntBiFunction`과 같다)
구현하는 쪽이 지켜야 할 규약이 있다	`compare` 결과의 부호 규칙
쓸모 있는 default 메서드를 붙일 수 있다	`Comparator.thenComparing`, `reversed`
직접 만들기로 했다면	`@FunctionalInterface`를 붙인다. 추상 메서드가 둘이 되면 컴파일러가 막는다

- 아이템 42: 익명 클래스보다는 람다를 사용하라
- 아이템 43: 람다보다는 메서드 참조를 사용하라 (이름만)
- 아이템 44: 표준 함수형 인터페이스를 사용하라

5교시

스트림처럼 생긴 반복문

`stream()` 을 썼다고 스트림답게 쓴 것일까?

부대에서 체력 50 넘는 유닛 이름을 모은다

메딕이 없어도 버틸 유닛을 고르는 보고서다. 처음엔 익숙한 `for`로 쓴다.

SquadReport.java

```
import java.util.ArrayList;
import java.util.List;

final class SquadReport {

    private SquadReport() {
    }

    static List<String> namesOfHealthy(List<Unit> units) {
        List<String> names = new ArrayList<>();
        for (Unit unit : units) {
            if (unit.getHp() > 50) {
                names.add(unit.getName());
            }
        }
        return names;
    }
}
```

`Squad`, `Unit`은 1·2교시 그대로다. 마린(40), SCV(60), 레이스(120) → `[SCV, Wraith]`

테스트: 이름 목록

SquadReportTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadReportTest {

    @Test
    @DisplayName("체력 50을 넘는 SCV와 레이스의 이름만 모은다")
    void collectNamesOfHealthyUnits() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(SCV.create());
        squad.add(Wraith.create());

        List<String> names = SquadReport.namesOfHealthy(squad.getUnits());

        assertEquals(Arrays.asList("SCV", "Wraith"), names);
    }
}
```

```
✓ 체력 50을 넘는 SCV와 레이스의 이름만 모은다
```

스트림으로 바꿔 봤다: forEach 안에서 names.add

4교시에 배운 람다로 「스트림답게」 바꿨다. `SquadReport.java`의 `namesOfHealthy()`를 이것으로 바꾼다. 테스트는 그대로 통과한다.

```
static List<String> namesOfHealthy(List<Unit> units) {
    List<String> names = new ArrayList<>();
    units.stream()
            .filter(u -> u.getHp() > 50)
            .forEach(u -> names.add(u.getName())); // 바깥 목록을 바꾼다
    return names;
}
```

모양은 스트림인데, 하는 일은 `for` 반복문과 똑같다. 스트림 바깥의 `names`를 람다 안에서 바꾼다.

> 현업 장면: 코드 리뷰에서 「forEach 안에서 바깥 리스트에 add하지 마세요」라는 지적을 받아 봤다면, 바로 이 코드다.

이 코드를 parallelStream으로 바꾸면 결과가 매번 다르다

SCV 5000, 마린 5000을 넣고 같은 파이프라인을 `parallelStream()`으로 다섯 번 돌린다.

ParallelSideEffectDemo.java

```
import java.util.ArrayList;
import java.util.List;

public class ParallelSideEffectDemo {

    public static void main(String[] args) {
        List<Unit> units = new ArrayList<>();
        for (int i = 0; i < 10_000; i++) {
            units.add(i % 2 == 0 ? SCV.create() : Marine.create()); // SCV 5000, 마린 5000
        }

        for (int run = 1; run <= 5; run++) {
            List<String> names = new ArrayList<>();
            try {
                units.parallelStream()
                        .filter(u -> u.getHp() > 50)
                        .forEach(u -> names.add(u.getName()));
                System.out.println(run + "회: " + names.size() + "개");
            } catch (RuntimeException e) {
                System.out.println(run + "회: " + e.getClass().getSimpleName());
            }
        }
    }
}
```

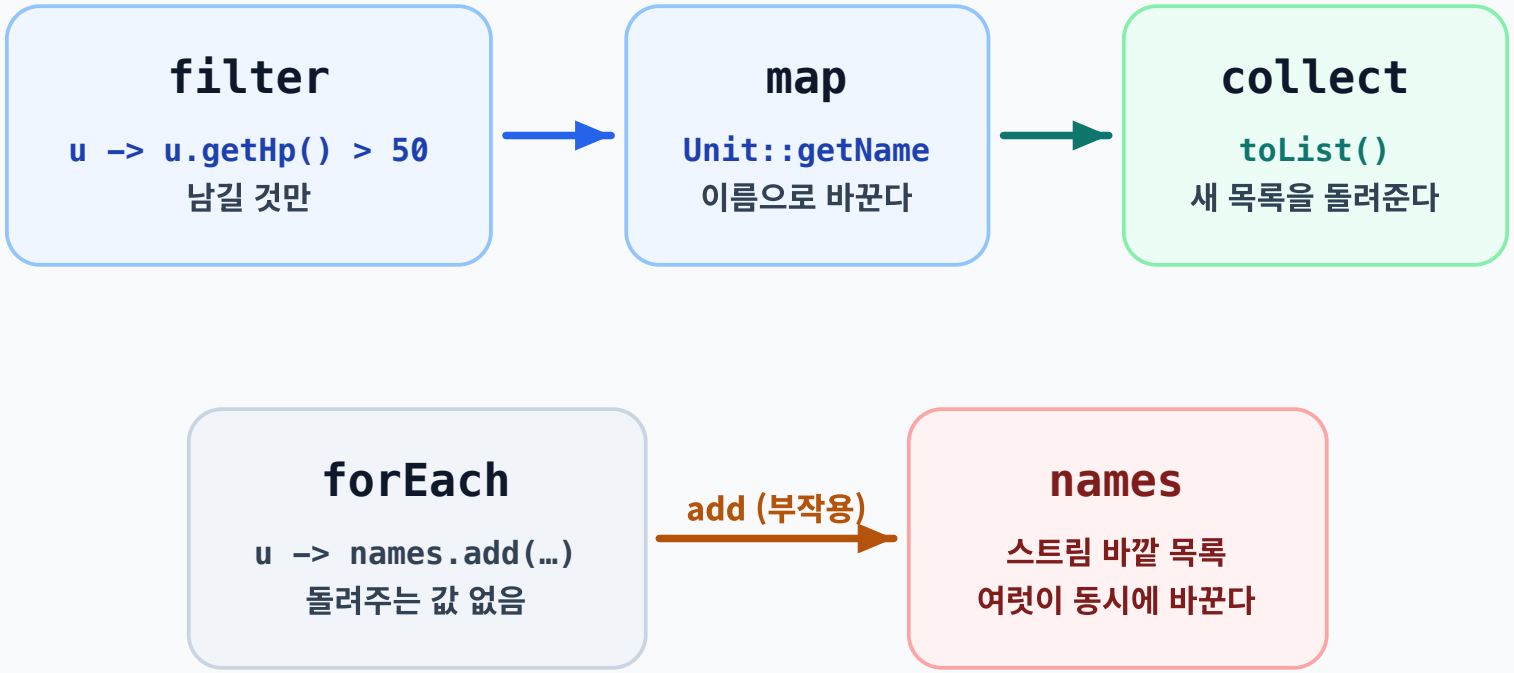
다섯 번 돌리면 다섯 번 다른 개수가 나온다

강사 PC 실행 예(매번 다름). 기대값은 SCV 5000개다.

```
1회: 1997개
2회: 976개
3회: 1058개
4회: 1083개
5회: 872개
```

여러 스레드가 `ArrayList` 하나에 동시에 `add` 하니 값이 사라진다. 예외가 날 때도 있다.

스트림의 일은 값을 계산해 돌려주는 것



위쪽은 단계마다 값을 받아 새 값을 넘기고, 마지막 `collect`가 결과를 **돌려준다**. 바깥 상자에는 손대지 않는다.
이것이 이펙티브 자바 아이템 46: 스트림에서는 부작용 없는 함수를 사용하라.

collect(Collectors.toList())로 바꾼다

`names`를 미리 만들지 않는다. 스트림이 만든 목록을 그대로 돌려준다.
`SquadReport.java`의 `namesOfHealthy()`를 이것으로 바꾸고,

파일 맨 위에 `import static java.util.stream.Collectors.toList;`를 더한다.

```
static List<String> namesOfHealthy(List<Unit> units) {
    return units.stream()
            .filter(u -> u.getHp() > 50)
            .map(Unit::getName)
            .collect(toList());
}
```

- `forEach`는 결과를 출력하거나 보고할 때만 쓴다. 계산에는 쓰지 않는다.
- `Collectors`의 메서드는 static import로 쓰는 것이 관례다. 그래야 파이프라인이 짧게 읽힌다.

`SquadReportTest`를 다시 돌리면 그대로 통과한다. 바깥 상태가 없으니 `parallelStream()`으로 바꿔도 값이 사라지지 않는다.

조건을 밖에서 받는다: namesOf(units, condition)

공중 유닛 이름, 다친 유닛 이름도 필요하다. 조건마다 `namesOfAir`, `namesOfWounded`를 만들 필요는 없다.
조건을 `Predicate<Unit>`으로 받아 `filter`에 그대로 넘긴다.
SquadReport.java에 메서드를 더하고, 파일 맨 위에 `import java.util.function.Predicate;`를 더한다.

```
static List<String> namesOf(List<Unit> units, Predicate<Unit> condition) {
    return units.stream()
            .filter(condition)
            .map(Unit::getName)
            .collect(toList());
}
```

조건은 호출하는 쪽이 고른다. 4교시에 만든 `TargetRules`를 그대로 넘긴다(발췌).

```
SquadReport.namesOf(units, TargetRules.isAir());
SquadReport.namesOf(units, TargetRules.isAir().and(TargetRules.isWounded()));
SquadReport.namesOf(units, u -> u.getHp() > 50); // namesOfHealthy와 같은 결과
```

테스트: TargetRules 조건을 넘겨 이름을 모은다

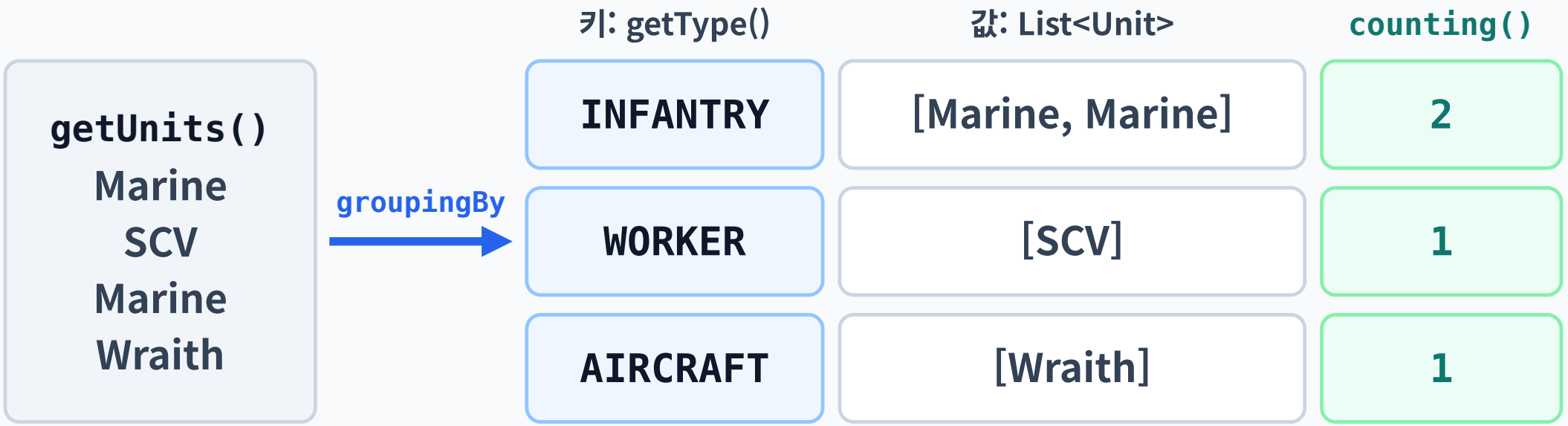
SquadReportTest.java 에 이 테스트를 더한다. 4교시에 만든 `TargetRules.java`가 같은 폴더에 있어야 한다.

```
@Test
@DisplayName("TargetRules 조건을 넘기면 그 조건에 맞는 유닛 이름만 모은다")
void collectNamesByCondition() {
    Squad squad = new Squad();
    squad.add(Marine.ofDamaged(20));
    squad.add(SCV.create());
    squad.add(Wraith.ofDamaged(30));
    squad.add(Wraith.create());
    List<Unit> units = squad.getUnits();

    assertEquals(Arrays.asList("Wraith", "Wraith"),
            SquadReport.namesOf(units, TargetRules.isAir()));
    assertEquals(Arrays.asList("Marine", "Wraith"),
            SquadReport.namesOf(units, TargetRules.isWounded()));
    assertEquals(Arrays.asList("Wraith"),
            SquadReport.namesOf(units, TargetRules.isAir().and(TargetRules.isWounded())));
}
```

```
✓ 체력 50을 넘는 SCV와 레이스의 이름만 모은다
✓ TargetRules 조건을 넘기면 그 조건에 맞는 유닛 이름만 모은다
```

groupingBy: 종류별로 묶고, counting으로 센다



`groupingBy(Unit::getType)`은 종류를 키로, 같은 종류 유닛 목록을 값으로 담는다.
`groupingBy`의 두 번째 인자는 칸 안을 줄이는 방법이다(`counting()`은 `Long`).

SquadReport에 byType, countByType을 더한다

`SquadReport.java`의 import 줄 아래에 세 줄을, 클래스 안에 두 메서드를 더한다.

```
// import 줄 아래에 더한다
import java.util.Map;
import static java.util.stream.Collectors.counting;
import static java.util.stream.Collectors.groupingBy;

// 클래스 안에 더한다
static Map<UnitType, List<Unit>> byType(List<Unit> units) {
    return units.stream().collect(groupingBy(Unit::getType));
}

static Map<UnitType, Long> countByType(List<Unit> units) {
    return units.stream().collect(groupingBy(Unit::getType, counting()));
}
```

`byType`은 그림의 「값」 열, `countByType`은 「counting()」 열을 만든다.

테스트: 종류별 개수

SquadReportTest.java 에 도우미와 테스트를 더하고, 파일 맨 위에 `import java.util.Map;`을 더한다.

```
private Squad mixedSquad() {
    Squad squad = new Squad();
    squad.add(Marine.create());
    squad.add(SCV.create());
    squad.add(Marine.create());
    squad.add(Wraith.create());
    return squad;
}

@Test
@DisplayName("마린 2, SCV 1, 레이스 1이면 종류별 개수는 2, 1, 1")
void countUnitsByType() {
    Map<UnitType, Long> counts = SquadReport.countByType(mixedSquad().getUnits());

    assertEquals(2L, counts.get(UnitType.INFANTRY));
    assertEquals(1L, counts.get(UnitType.WORKER));
    assertEquals(1L, counts.get(UnitType.AIRCRAFT));
}
```

```
✓ 체력 50을 넘는 SCV와 레이스의 이름만 모은다
✓ TargetRules 조건을 넘기면 그 조건에 맞는 유닛 이름만 모은다
✓ 마린 2, SCV 1, 레이스 1이면 종류별 개수는 2, 1, 1
```

실습 준비: SquadReportLabTest를 먼저 붙여 넣는다

실습 SquadReportLabTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadReportLabTest {

    @Test
    @DisplayName("마린 40 + 40, SCV 60이면 INFANTRY=80, WORKER=60")
    void sumHpByType() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(Marine.create());
        squad.add(SCV.create());

        Map<UnitType, Integer> totals = SquadReport.totalHpByType(squad.getUnits());

        assertEquals(80, totals.get(UnitType.INFANTRY));
        assertEquals(60, totals.get(UnitType.WORKER));
        assertEquals(2, totals.size());
    }
}
```

종류별 체력 합계 (실습)

실습

1. 앞에서 만든 `SquadReport.java` 에 시작 코드를 더한다.

```
static Map<UnitType, Integer> totalHpByType(List<Unit> units) {
    throw new UnsupportedOperationException("실습: totalHpByType");
}
```

2. `SquadReportLabTest` 를 실행한다. `UnsupportedOperationException: 실습: totalHpByType` 으로 실패한다.
3. `countByType` 처럼 `groupingBy` 로 칸을 나누고, 칸 안은 체력을 더하게 고친다(`Collectors.summingInt`).
4. 다시 실행해 초록을 본다.

```
✓ 마린 40 + 40, SCV 60이면 INFANTRY=80, WORKER=60
```

더 해 보기: `averagingInt(Unit::getHp)` 로 종류별 평균 체력을 구해 보자.

실습 풀이: counting() 자리만 summingInt로 바꾼다

풀이

`SquadReport.java`의 `totalHpByType()`을 이것으로 바꾸고, 파일 맨 위에 `import static java.util.stream.Collectors.summingInt;`를 더한다.

```
static Map<UnitType, Integer> totalHpByType(List<Unit> units) {
    return units.stream()
            .collect(groupingBy(Unit::getType, summingInt(Unit::getHp)));
}
```

칸을 나누는 일(`groupingBy`)과 칸 안을 줄이는 일(`counting`, `summingInt`)이 따로 조립된다.

반대로, 전부 스트림으로 쓰면

메딕 지원 요청서: 다친 유닛(체력 50 미만)이 **둘 이상** 몰린 종류만, 「이름(체력)」을 쉼표로 잇는다.
`SquadReport.java` 에 메서드를 더하고, 파일 맨 위에

`import static java.util.stream.Collectors.joining;`, `import static java.util.stream.Collectors.toMap;`을 더한다.

```
static Map<UnitType, String> woundedSummary(List<Unit> units) {
    return units.stream()
            .collect(groupingBy(Unit::getType))
            .entrySet().stream()
            .filter(e -> e.getValue().stream()
                    .filter(u -> u.getHp() < 50)
                    .count() >= 2)
            .collect(toMap(e -> e.getKey(),
                    e -> e.getValue().stream()
                            .filter(u -> u.getHp() < 50)
                            .map(u -> u.getName() + "(" + u.getHp() + ")")
                            .collect(joining(", "))));
}
```

마린 20, 마린 30, SCV 10, 레이스 120이면 `{INFANTRY=Marine(20), Marine(30)}`이 나와야 한다.

테스트: 메딕 지원 요청서

WoundedSummaryTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Collections;
import java.util.List;
import java.util.Map;

import static org.junit.jupiter.api.Assertions.assertEquals;

class WoundedSummaryTest {

    @Test
    @DisplayName("다친 유닛이 둘 이상인 종류만 이름(체력)을 잇는다")
    void summarizeTypesWithTwoOrMoreWounded() {
        Squad squad = new Squad();
        squad.add(Marine.ofDamaged(20));
        squad.add(Marine.ofDamaged(30));
        squad.add(SCV.ofDamaged(10));
        squad.add(Wraith.create());

        Map<UnitType, String> summary = SquadReport.woundedSummary(squad.getUnits());

        assertEquals(Collections.singletonMap(UnitType.INFANTRY, "Marine(20), Marine(30)"), summary);
    }
```

테스트: 메딕 지원 요청서 (이어서)

`WoundedSummaryTest.java` (이어서)

```
    @Test
    @DisplayName("다친 유닛이 없으면 빈 요청서다")
    void emptyWhenNobodyWounded() {
        List<Unit> units = Collections.<Unit>singletonList(Wraith.create());

        assertEquals(Collections.emptyMap(), SquadReport.woundedSummary(units));
    }
}
```

```
✓ 다친 유닛이 둘 이상인 종류만 이름(체력)을 잇는다
✓ 다친 유닛이 없으면 빈 요청서다
```

2건 모두 통과한다. 동작은 맞다. 그런데 앞 장의 `woundedSummary`는 읽기도 디버깅도 어렵다.

- 단계마다 이름이 없어서 `e`, `u`가 무엇인지 매번 거슬러 올라가야 한다.
- 「다쳤다」 조건(`getHp() < 50`)이 두 번 들어 있다. 기준이 바뀌면 두 곳을 고쳐야 한다.
- 중간 값을 보려고 해도 변수가 없어 브레이크포인트를 걸 곳이 없다.

스트림을 쓰면 무조건 좋아지는 것이 아니라면, 무엇을 기준으로 골라야 할까?

스트림과 반복문을 고르는 기준

스트림이 유리하다	반복문이 유리하다
원소를 일관되게 바꾼다(`map`)	바깥 지역 변수를 바꿔야 한다(람다 안에서는 못 바꾼다)
조건으로 걸러 낸다(`filter`)	중간에 `break`, `continue`, `return` 해야 한다
합치거나 묶는다(`collect`, `groupingBy`)	검사 예외(`IOException` 등)를 던지는 메서드를 부른다
조건에 맞는 원소를 찾는다(`anyMatch`, `findFirst`)	앞 단계의 값과 지금 값을 함께 써야 한다

이것이 이펙티브 자바 아이템 45: 스트림은 주의해서 사용하라.

> 둘 중 하나만 고르는 문제가 아니다. 한 메서드 안에서 스트림과 반복문을 섞고, 단계마다 이름을 붙이는 것이 가장 읽기 좋은 경우가 많다.

헬퍼 메서드로 단계에 이름을 붙인다

길게 이어진 람다를 「무엇을 하는가」라는 이름의 메서드로 뺀다. `SquadReport.java`에 헬퍼 세 개를 더한다.

```
private static Map<UnitType, List<Unit>> woundedByType(List<Unit> units) {
    return units.stream()
            .filter(TargetRules.isWounded())
            .collect(groupingBy(Unit::getType));
}

private static String describe(List<Unit> units) {
    return units.stream()
            .map(SquadReport::label)
            .collect(joining(", "));
}

private static String label(Unit unit) {
    return unit.getName() + "(" + unit.getHp() + ")";
}
```

「다쳤다」 조건은 4교시 실습의 `TargetRules.isWounded()`(`WOUNDED_HP = 50` 미만) 한 곳에만 있다.

본문은 이름만 읽힌다

`Map`을 다시 스트림으로 돌리는 대신, 칸을 하나씩 보는 일은 반복문에 맡긴다.
`SquadReport.java`의 `woundedSummary()`를 이것으로 바꾸고, 파일 맨 위에 `import java.util.HashMap;`을 더한다.

```
static Map<UnitType, String> woundedSummary(List<Unit> units) {
    Map<UnitType, List<Unit>> wounded = woundedByType(units);
    Map<UnitType, String> summary = new HashMap<>();
    for (Map.Entry<UnitType, List<Unit>> entry : wounded.entrySet()) {
        if (entry.getValue().size() >= 2) {
            summary.put(entry.getKey(), describe(entry.getValue()));
        }
    }
    return summary;
}
```

```
✓ 다친 유닛이 둘 이상인 종류만 이름(체력)을 잇는다
✓ 다친 유닛이 없으면 빈 요청서다
```

같은 `WoundedSummaryTest`가 그대로 통과한다. `wounded`에 브레이크포인트를 걸면 중간 결과도 보인다.

이 코드는 스트림과 반복문 중 무엇이 나을까? (퀴즈)

퀴즈

각 작업에 스트림과 반복문 중 무엇이 나을지 고르세요.

번호	작업
1	부대 유닛 이름을 모두 대문자로 바꿔 목록으로 돌려준다
2	유닛마다 `ReplayWriter.write(...)`로 기록한다(`IOException`을 던진다)
3	앞에서부터 유닛을 넣다가 체력 합이 200을 넘는 순간 멈춘다
4	종류별로 가장 체력이 낮은 유닛을 찾는다

풀이: 변환·집계는 스트림, 흐름 제어·검사 예외는 반복문

정답

번호	답	이유
1	스트림	`map(u -> u.getName().toUpperCase())` + `collect(toList())` 한 줄
2	반복문	람다 안에서는 검사 예외를 바깥으로 던질 수 없다. 1일차 `ReplayWriter`가 그렇다
3	반복문	누적 합(지역 변수)과 `break`가 필요하다. Java 8에는 `takeWhile`도 없다
4	스트림	`groupingBy(Unit::getType, minBy(comparingInt(Unit::getHp)))`

> 고르는 기준은 「무엇이 더 멋있나」가 아니라 「다음 사람이 무엇을 더 빨리 읽나」다. 애매하면 둘 다 써 보고 읽기 쉬운 쪽을 남긴다.

느린 외부 호출을 parallelStream으로 빨리?

유닛마다 외부 가격 API(1초)를 부른다. 열 번이면 10초라서 `parallelStream()`을 붙였다.

강사 시연(따라 치지 않는다) · 발췌: `CommonPoolDemo`

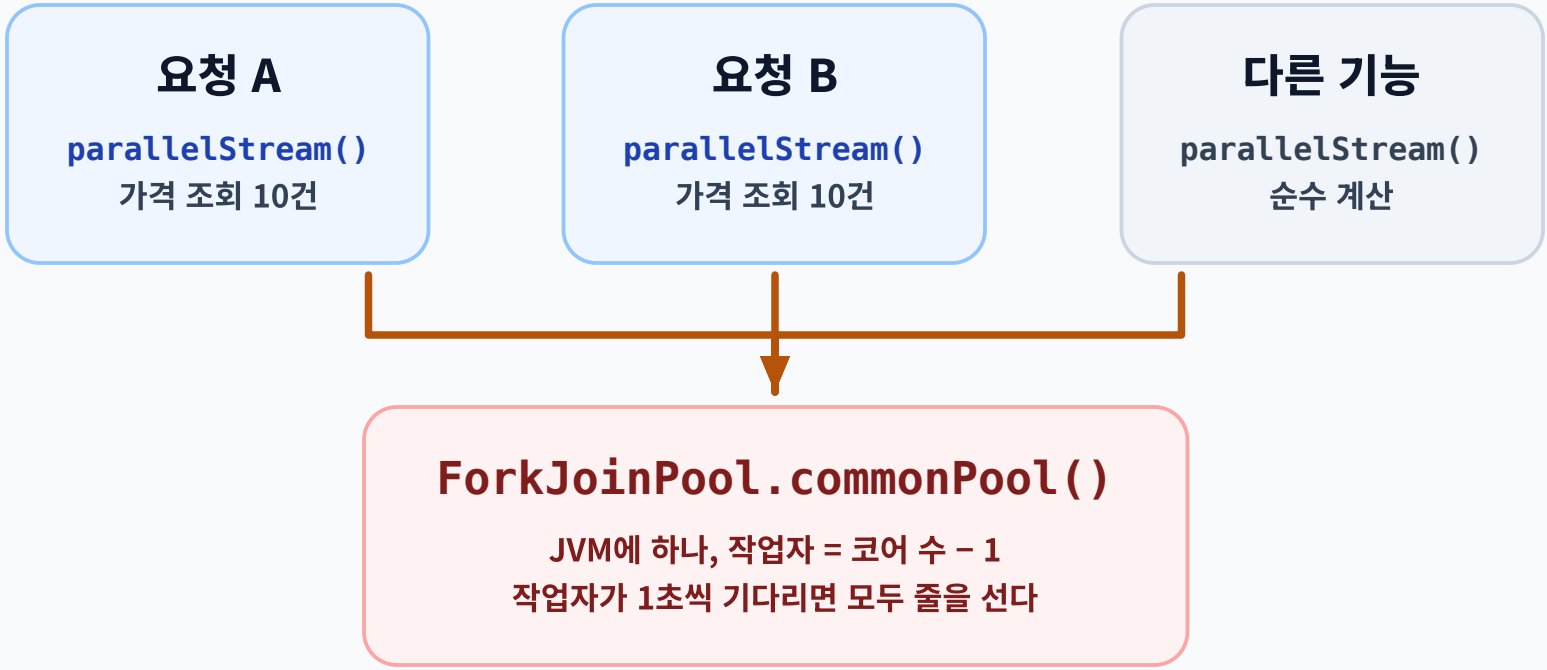
```
static List<Integer> pricesOf(List<Unit> units) {
    return units.parallelStream()
            .map(CommonPoolDemo::fetchPrice) // 안에서 Thread.sleep(1000)
            .collect(toList());
}
// main: 마린 10기로 한 번 재고, 이어서 스레드 둘이 동시에 pricesOf를 부를 때를 잰다
```

```
공용 풀 작업자 수: 9
요청 하나: 1.012초
요청 둘 동시: 2.017초
```

강사 PC 측정값(10코어, 매번 다름). 혼자일 땐 10초가 1초가 됐는데, 요청이 둘이면 각자 2초다.

> 현업 장면: 웹 요청마다 parallelStream으로 외부 API를 불러 서버 전체가 느려졌다.

모든 병렬 스트림은 공용 풀 하나를 나눠 쓴다



`parallelStream()`은 JVM에 하나뿐인 공용 풀을 빌려 쓴다. 한 요청이 붙잡으면 다른 기능도 줄을 선다.
이것이 이펙티브 자바 아이템 48: 스트림 병렬화는 주의해서 적용하라.

병렬화 도입 체크리스트

`parallelStream()`을 붙이기 전에 네 가지를 모두 확인한다. 하나라도 아니면 순차 스트림으로 둔다.

확인할 것	병렬이 이득인 경우	손해인 경우
원소 수와 원소당 계산량	수십만 건 이상의 CPU 계산	수십 건, 계산이 가벼움
원천을 고르게 나눌 수 있나	`ArrayList`, 배열, `IntStream.range`	`LinkedList`, `Stream.iterate`, I/O 원천
상태 공유가 없나	부작용 없는 `map` · `filter` · `collect`	바깥 목록에 `add`, 공유 필드 수정
측정으로 확인했나	운영과 같은 조건에서 빨라졌다	「빨라질 것 같아서」

> I/O를 기다리는 작업(외부 API, DB)은 CPU 계산이 아니다. 병렬 스트림이 아니라 크기를 정한 별도 스레드 풀의 일이다.

구두 질문: 요청마다 DB 20건 조회 vs 야간 배치 100만 건 계산, 병렬 후보는? (배치, 측정 후)

정리: 스트림 세 가지 원칙

아이템	오늘의 한 줄
46 부작용 없는 스트림	`forEach`로 바깥을 바꾸지 말고 `collect`로 돌려받는다
45 스트림은 주의해서	변환·집계는 스트림, 흐름 제어·검사 예외는 반복문, 섞어도 된다
48 병렬화는 주의해서	공용 풀 하나를 나눠 쓴다. 체크리스트 네 개와 측정

6교시

if (list != null)이 퍼진 코드

「없음」을 null로 알리면, 그 null은 누가 처리할까?

빈 부대의 getUnits는 null을 돌려준다

앞 교시의 `Squad`에 누군가 「빈 부대면 없다고 알려 주자」며 한 줄을 넣었다.
앞에서 만든 Squad.java의 `getUnits()`를 이것으로 바꾼다.

```
List<Unit> getUnits() {
    if (units.isEmpty()) {
        return null; // 빈 부대는 "없음"이라고 알리려는 의도
    }
    return new ArrayList<>(units);
}
```

의도는 친절하다. 이 `null`을 받는 쪽을 보자.

호출부마다 null 검사, 하나만 빠뜨려도 NPE

`EmptySquadTest.java` 빈 부대를 두 호출부가 받는다. A는 검사했고 B는 빠뜨렸다.

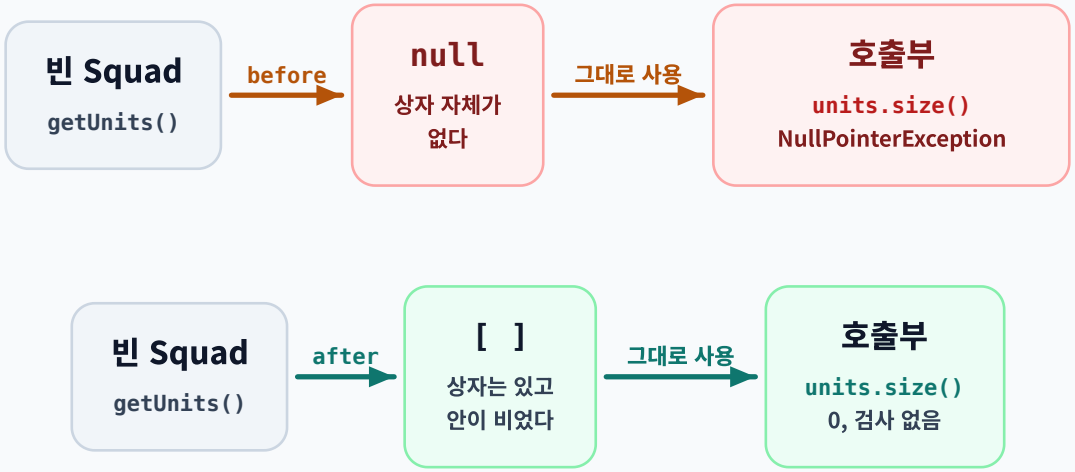
```
import java.util.List;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class EmptySquadTest {
    @Test
    @DisplayName("null 검사를 한 호출부는 빈 부대에서 0을 센다")
    void countWithNullCheck() {          // 호출부 A
        List<Unit> units = new Squad().getUnits();
        int count = 0;
        if (units != null) {
            count = units.size();
        }
        assertEquals(0, count);
    }

    @Test
    @DisplayName("빈 부대의 유닛 수는 0이다")
    void emptySquadHasNoUnits() {        // 호출부 B
        List<Unit> units = new Squad().getUnits();
        int count = units.size(); // null 검사를 빠뜨렸다
        assertEquals(0, count);
    }
}
```

결과: A는 ✓ 통과, B는 ✘ `java.lang.NullPointerException`. 호출부가 열 곳이면 `if` 도 열 개다.

없음은 '빈 목록'으로 말한다



`null`은 상자 자체가 없는 것이고, 빈 목록은 상자는 있되 안이 빈 것이다. 빈 목록은 `size()`, `for`, 스트림이 그대로 동작한다.
이펙티브 자바 아이템 54: null이 아닌, 빈 컬렉션이나 배열을 반환하라.

> 현업 장면: DAO가 결과가 없을 때 `null`을 돌려줘서 서비스마다 `if (list != null)`이 퍼진 코드가 이 모양이다.

복사본 반환 한 줄이면 빈 부대도 해결된다

앞에서 만든 Squad.java 의 `getUnits()` 를 이것으로 바꾸고, `import java.util.Collections;` 를 더한다.

```
List<Unit> getUnits() {
    return Collections.unmodifiableList(new ArrayList<>(units));
}
```

- `units` 가 비어 있으면 복사본도 빈 목록이다. 빈 경우를 따로 가를 필요가 없다.
- `new ArrayList<>(units)` 로 복사하니 나중에 부대에 유닛이 늘어도 받은 목록은 그대로다.
- `unmodifiableList` 로 감싸니 받은 쪽이 목록을 고칠 수 없다(앞 교시의 복사본보다 한 단계 더).

> 빈 부대에서 객체를 만드는 비용까지 아끼고 싶다면 `if (units.isEmpty()) return Collections.emptyList();` 를 앞에 둘 수 있다. 측정해서 필요할 때만 쓰는 최적화다.

테스트: 빈 부대의 getUnits().size()는 0

null 검사를 빠뜨렸던 호출부 B(`emptySquadHasNoUnits`)가 고치지 않고도 통과한다. 복사본·읽기 전용도 확인하자. EmptySquadTest.java 에 `import static org.junit.jupiter.api.Assertions.assertThrows;` 와 아래 두 테스트를 더한다.

```
@Test
@DisplayName("받은 목록에 유닛을 넣으면 UnsupportedOperationException")
void returnedListIsUnmodifiable() {
    Squad squad = new Squad();
    squad.add(Marine.create());
    List<Unit> units = squad.getUnits();
    assertThrows(UnsupportedOperationException.class, () -> units.add(SCV.create()));
}

@Test
@DisplayName("목록을 받은 뒤 부대에 추가해도 받은 목록은 그대로다")
void returnedListIsSnapshot() {
    Squad squad = new Squad();
    squad.add(Marine.create());
    List<Unit> units = squad.getUnits();
    squad.add(SCV.create());
    assertEquals(1, units.size());
}
```

실행하면 4건 모두 ✓. 고친 곳은 `Squad` 한 곳이고, 호출부는 하나도 고치지 않았다.

이름으로 유닛을 찾는다: 없으면 null

이번에는 목록이 아니라 유닛 **하나**를 돌려준다. 빈 목록 같은 「빈 유닛」은 없다.

앞에서 만든 Squad.java 에 `findByName()` 을 더한다.

```
Unit findByName(String name) {
    for (Unit unit : units) {
        if (unit.getName().equals(name)) {
            return unit;
        }
    }
    return null; // 못 찾으면 null
}
```

시그니처만 보면 `Unit findByName(String name)` 이다. 못 찾을 수 있다는 말은 어디에도 없다.

반환 타입만 봐서는 null이 올지 모른다

`FindByNameTest.java` 마린과 SCV만 있는 부대에서 SCV와 메딕을 찾는다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

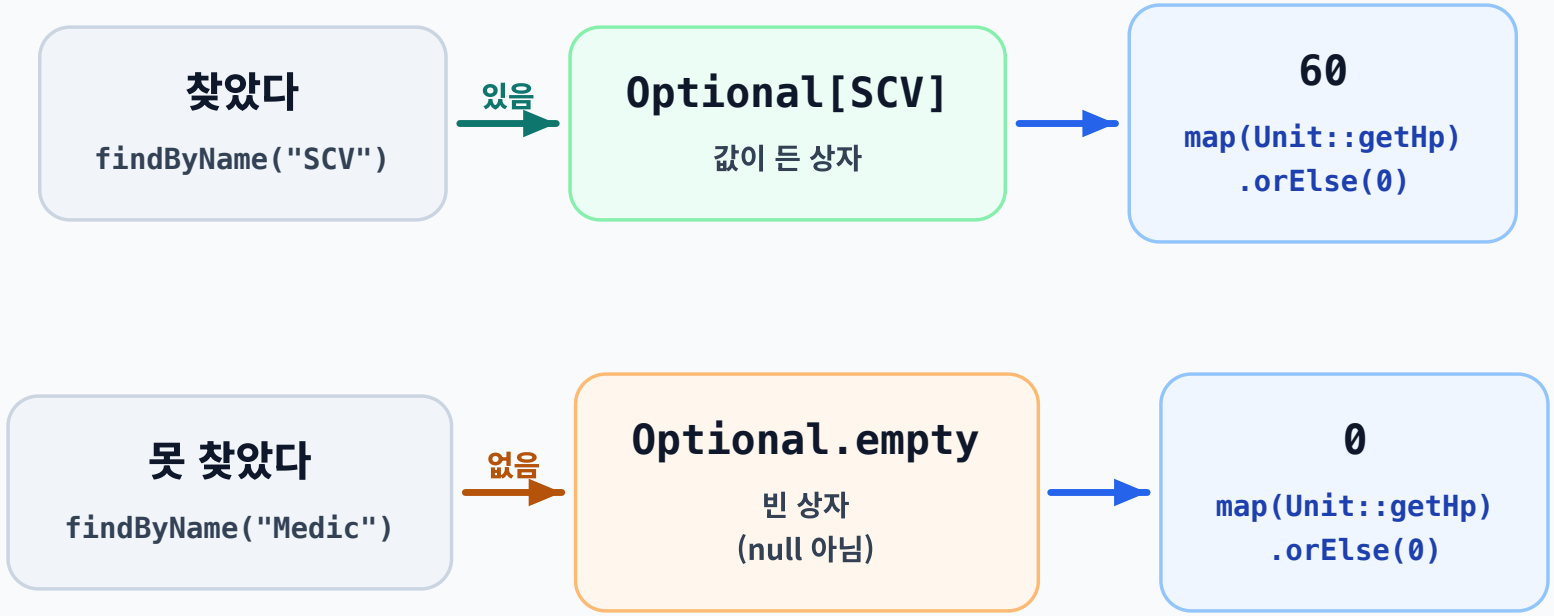
class FindByNameTest {
    private Squad marineAndScv() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(SCV.create());
        return squad;
    }

    @Test
    @DisplayName("부대에 있는 SCV를 이름으로 찾는다")
    void findScv() {
        Unit scv = marineAndScv().findByName("SCV");
        assertEquals(60, scv.getHp());
    }

    @Test
    @DisplayName("메딕이 없으면 메딕 체력은 0으로 본다")
    void missingMedicHpIsZero() {
        int medicHp = marineAndScv().findByName("Medic").getHp(); // 반환 타입은 Unit
        assertEquals(0, medicHp);
    }
}
```

결과: SCV 찾기 ✓, 메딕 ✘ `java.lang.NullPointerException`. 호출한 사람은 null이 온다는 것을 **알 길이 없었다**.

'없을 수도 있음'을 타입에 적는다: Optional



`Optional<Unit>`은 유닛이 들었을 수도, 비었을 수도 있는 상자다. 반환 타입만 봐도 「없을 수 있다」가 보인다.
이펙티브 자바 아이템 55: 옵셔널 반환은 신중히 하라.

Optional<Unit> findByName 구현

찾으면 `Optional.of(unit)`, 못 찾으면 `Optional.empty()`. `null`은 어디에도 없다.

앞에서 만든 Squad.java의 `findByName()`을 이것으로 바꾸고, `import java.util.Optional;`을 더한다.

```
Optional<Unit> findByName(String name) {
    for (Unit unit : units) {
        if (unit.getName().equals(name)) {
            return Optional.of(unit);
        }
    }
    return Optional.empty();
}
```

반환 타입이 바뀌었으니 `FindByNameTest`는 이제 컴파일되지 않는다. 호출부가 「없으면 어떻게 할지」를 적어야 한다.

호출부는 「없으면 0」을 직접 적는다

`FindByNameTest.java` 에 `import static org.junit.jupiter.api.Assertions.assertFalse;` 를 더하고,

테스트 두 개를 이것으로 바꾸고 하나를 더한다. 도우미 `marineAndScv()` 는 그대로 둔다.

```
@Test
@DisplayName("부대에 있는 SCV를 이름으로 찾는다")
void findScv() {
    int scvHp = marineAndScv().findByName("SCV").map(Unit::getHp).orElse(0);
    assertEquals(60, scvHp);
}

@Test
@DisplayName("없는 이름을 찾으면 빈 Optional이 온다")
void missingNameGivesEmpty() {
    assertFalse(marineAndScv().findByName("Medic").isPresent());
}

@Test
@DisplayName("메딕이 없으면 메딕 체력은 0으로 본다")
void missingMedicHpIsZero() {
    int medicHp = marineAndScv().findByName("Medic").map(Unit::getHp).orElse(0);
    assertEquals(0, medicHp);
}
```

3건 모두 ✓. `map(Unit::getHp)` 는 「있으면 체력으로 바꾼다」, `orElse(0)` 은 「없으면 0」이다.

꺼내는 방법들: orElse, orElseGet, orElseThrow, map, ifPresent

모두 Java 8에 있다.

메서드	값이 없을 때	예
`orElse(값)`	준비해 둔 값	`.orElse(Marine.create())`
`orElseGet(Supplier)`	그때 만든 값	`.orElseGet(Marine::create)`
`orElseThrow(Supplier)`	내가 고른 예외	`.orElseThrow(() -> new NoSuchElementException("..."))`
`map(Function)`	빈 Optional 그대로	`.map(Unit::getName)`
`ifPresent(Consumer)`	아무것도 안 함	`.ifPresent(u -> log.add(u.getName()))`

> Java 8에는 `isEmpty()`(11), 인자 없는 `orElseThrow()`(10), `ifPresentOrElse`(9)가 없다. 「비었나?」는 `!isPresent()`로 묻는다.

값이 있어도 orElse(createDefaultUnit())의 인자는 실행될까? (퀴즈)

퀴즈

부대에 SCV가 **있다**. `createDefaultUnit()`은 「기본 유닛 생성!」을 출력하고 마린을 만든다.

```
Squad squad = new Squad();
squad.add(SCV.create());

Unit a = squad.findByName("SCV").orElse(createDefaultUnit());
```

1. 실행되지 않는다. SCV가 있으니 `orElse`가 인자를 쓰지 않는다
2. 실행된다. 「기본 유닛 생성!」이 출력되고, `a`는 SCV다
3. 실행된다. 「기본 유닛 생성!」이 출력되고, `a`는 마린이다

풀이: 실행된다 — 인자는 호출 전에 먼저 계산된다

정답

정답은 2번이다. 직접 돌려 본다. OrElseDemo.java

```
public class OrElseDemo {
    static Unit createDefaultUnit() {
        System.out.println("  기본 유닛 생성!");
        return Marine.create();
    }

    public static void main(String[] args) {
        Squad squad = new Squad();
        squad.add(SCV.create());

        System.out.println("orElse 호출");
        Unit a = squad.findByName("SCV").orElse(createDefaultUnit());
        System.out.println("  찾은 유닛: " + a.getName());

        System.out.println("orElseGet 호출");
        Unit b = squad.findByName("SCV").orElseGet(() -> createDefaultUnit());
        System.out.println("  찾은 유닛: " + b.getName());
    }
}
```

orElse의 인자는 값이 있어도 만들어진다

`OrElseDemo` 출력의 앞부분:

```
orElse 호출
  기본 유닛 생성!
  찾은 유닛: SCV
```

`orElse(createDefaultUnit())`는 평범한 메서드 호출이다. 자바는 메서드를 부르기 전에 인자를 먼저 계산하므로, `createDefaultUnit()`이 먼저 돌고 그 결과가 `orElse`에 들어간다. SCV가 있으니 만든 마린은 그냥 버려진다.

> 기본값이 상수(`orElse(0)`, `orElse("없음")`)라면 문제없다. 만드는 데 비용이 들거나 부작용(DB 조회, 로그, 카운트)이 있으면 문제다.

orElseGet은 필요할 때만 만든다



`orElseGet`은 유닛이 아니라 **만드는 방법**(`Supplier`, 4교시)을 받는다. 같은 데모의 뒷부분:

```
orElseGet 호출
  찾은 유닛: SCV
```

준비 코드: 만든 횟수를 세는 공장과 SquadService

CountingUnitFactory.java 테스트용 공장. `create()`를 부를 때마다 센다.

```
class CountingUnitFactory {
    private int createdCount;
    Unit create() {
        createdCount++;
        return Marine.create();
    }
    int getCreatedCount() {
        return createdCount;
    }
}
```

SquadService.java 실습에서 고칠 파일. 공장을 생성자로 받는다(아이템 5, 1일차).

```
class SquadService {
    private final Squad squad;
    private final CountingUnitFactory factory;
    SquadService(Squad squad, CountingUnitFactory factory) {
        this.squad = squad;
        this.factory = factory;
    }

    Unit findOrDefault(String name) {
        return squad.findByName(name).orElse(factory.create());
    }
}
```

실습 테스트: 만들었는지를 숫자로 확인한다

SquadServiceTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class SquadServiceTest {
    private final Squad squad = new Squad();
    private final CountingUnitFactory factory = new CountingUnitFactory();
    private final SquadService service = new SquadService(squad, factory);

    @Test
    @DisplayName("찾는 유닛이 있으면 기본 유닛을 만들지 않는다")
    void noDefaultWhenFound() {
        squad.add(SCV.create());
        Unit unit = service.findOrDefault("SCV");
        assertEquals("SCV", unit.getName());
        assertEquals(0, factory.getCreatedCount());
    }

    @Test
    @DisplayName("찾는 유닛이 없으면 기본 유닛(마린)을 한 번 만든다")
    void defaultWhenMissing() {
        squad.add(SCV.create());
        Unit unit = service.findOrDefault("Medic");
        assertEquals("Marine", unit.getName());
        assertEquals(1, factory.getCreatedCount());
    }
}
```

JUnit은 테스트마다 테스트 객체를 새로 만드므로 필드의 부대·공장도 테스트마다 새것이다.

CountingUnitFactory로 생성 횟수 0을 확인한다 (실습)

실습

파일 수정: 앞에서 만든 `SquadService.java`

```
Unit findOrDefault(String name) {
    return squad.findByName(name).orElse(factory.create());
}
```

실행: `SquadServiceTest`. 지금은 한 건이 실패한다.

```
✘ 찾는 유닛이 있으면 기본 유닛을 만들지 않는다 → expected: <0> but was: <1>
✓ 찾는 유닛이 없으면 기본 유닛(마린)을 한 번 만든다
```

`orElse(...)`를 `orElseGet(factory::create)`(또는 `() -> factory.create()`)로 바꾸고 다시 실행한다.
기대 결과: 2건 모두 초록. 가짜 공장을 직접 만들어 넣었기에 「만들지 않았다」를 숫자로 확인할 수 있다.

Optional을 쓰지 않는 곳: 필드, 매개변수, 컬렉션 감싸기

`Optional`은 「반환값이 없을 수 있다」를 알리려고 만든 타입이다. 그 밖의 자리에서는 비용만 는다.

자리	쓰지 않는 이유	대신
필드	직렬화되지 않고, 필드가 또 null일 수 있다	평범한 필드 + 반환할 때 `Optional.ofNullable`
매개변수	호출부마다 `Optional.ofNullable(...)`로 싸야 한다	필수 인자 + `requireNonNull`, 또는 메서드 오버로딩
컬렉션·배열	「빈 Optional」과 「빈 목록」 두 가지 없음이 생긴다	빈 컬렉션(아이템 54)
`int` 같은 기본 타입	`Optional<Integer>`는 박싱한다	`OptionalInt`, `OptionalLong`, `OptionalDouble`

> 현업 장면: `Optional`을 엔티티 필드나 서비스 메서드 파라미터로 쓰고, 받자마자 `get()`을 부르던 코드. 타입만 바뀌었지 null 검사와 다를 게 없다.

get()을 바로 부르지 않는다

`get()`은 값이 없으면 터진다. null 대신 다른 예외가 날 뿐이다. 비교용 발췌(따라 치지 않는다):

```
// before: 값이 없으면 NoSuchElementException: No value present
Unit medic = squad.findByName("Medic").get();

// before: isPresent() + get()은 null 검사를 이름만 바꾼 것
Optional<Unit> found = squad.findByName("Medic");
if (found.isPresent()) {
    log.add(found.get().getName());
}
```

```
// after: 없을 때 무엇을 할지 적는다
int medicHp = squad.findByName("Medic").map(Unit::getHp).orElse(0);
squad.findByName("Medic").ifPresent(u -> log.add(u.getName()));
Unit medic = squad.findByName("Medic")
        .orElseThrow(() -> new IllegalStateException("메딕이 없습니다."));
```

반드시 있어야 하는 값이면 `orElseThrow`로 **무엇이 없는지** 메시지에 적는다.

이 코드에서 Optional 오용 세 군데를 찾으세요 (퀴즈)

퀴즈

MedicStation.java

```
import java.util.List;
import java.util.Optional;

class MedicStation {
    private Optional<Unit> patient = Optional.empty();   // A

    void admit(Optional<Unit> unit) {                    // B
        patient = unit;
    }

    Optional<Unit> currentPatient() {                    // C
        return patient;
    }

    Optional<List<Unit>> waitingList() {                 // D
        return Optional.empty();
    }
}
```

A~D 중 오용이 **아닌** 하나는?

풀이: 반환할 때만 Optional, 목록은 빈 목록

정답

오용은 A(필드), B(매개변수), D(목록 감싸기)다. C(반환값)만 제자리다. `MedicStation.java` 를 이것으로 바꾼다.

```
import java.util.Collections;
import java.util.List;
import java.util.Objects;
import java.util.Optional;

class MedicStation {
    private Unit patient;                                  // A: null 가능, 클래스 안에서만

    void admit(Unit unit) {                                // B: 필수 인자
        patient = Objects.requireNonNull(unit, "unit");
    }

    Optional<Unit> currentPatient() {                      // C: 그대로
        return Optional.ofNullable(patient);
    }

    List<Unit> waitingList() {                             // D: 빈 목록
        return Collections.emptyList();
    }
}
```

`null` 은 클래스 안에 가두고, 바깥으로 나갈 때 `Optional.ofNullable` 로 바꾼다.

정리: 「없음」을 돌려주는 법

돌려줄 것	없을 때	꺼내는 쪽
컬렉션·배열	빈 컬렉션(`unmodifiableList(new ArrayList<>(...))`)	검사 없이 `size()` , `for`
값 하나	`Optional.empty()`	`map` , `orElse` , `orElseGet` , `orElseThrow` , `ifPresent`
기본값을 만드는 데 비용·부작용이 있다	—	`orElseGet(Supplier)`
필드·매개변수	`Optional` 쓰지 않음	반환할 때만 `Optional.ofNullable`

- 아이템 54: null이 아닌, 빈 컬렉션이나 배열을 반환하라
- 아이템 55: 옵셔널 반환은 신중히 하라

7교시

게임에서 주문으로: 1원이 어긋난 정산

Day 1에 만든 Book으로 주문 금액을 계산하고 영수증을 찍는다.

0.1 + 0.2를 출력하면 무엇이 나올까? (퀴즈)

퀴즈

```
public class DoubleSumDemo {
    public static void main(String[] args) {
        System.out.println(0.1 + 0.2);
    }
}
```

1. `0.3`
2. `0.30000000000000004`
3. `0.30000001`
4. 컴파일 에러

0.30000000000000004

정답

정답은 2번이다. 실제로 실행한 결과다.

```
0.30000000000000004
```

버그가 아니라 `double`이 원래 이렇게 동작한다. 소수 열일곱째 자리의 오차, 넘어가도 될까?

> 화면에 보여 줄 숫자라면 반올림으로 가릴 수 있다. 그런데 이 값이 금액이라면?

금액이라면: 30% 할인가가 1원 모자란다

정가 22,000원 책을 30% 할인한다. 정답은 15,400원이다.

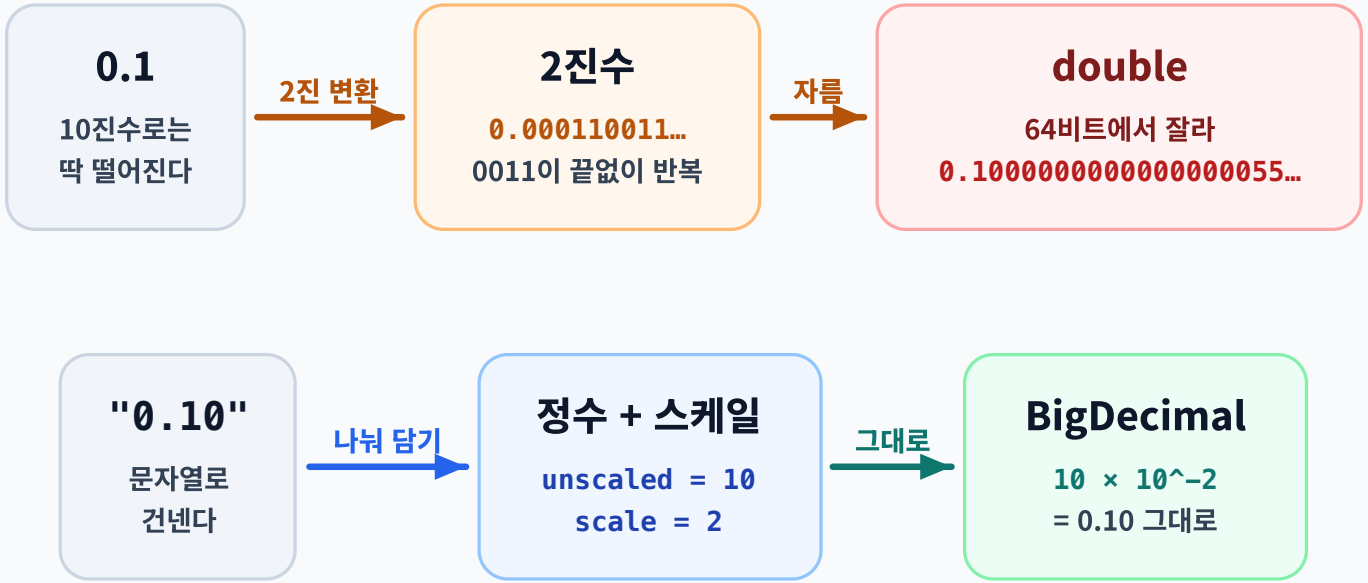
```
int price = 22000;                          // 정가
double rate = 0.7;                          // 30% 할인
long discounted = (long) (price * rate); // 원 미만 버림

System.out.println(price * rate);
System.out.println(discounted + "원");
```

```
15399.999999999998
15399원
```

> 현업에서 본 장면: `double`로 금액을 계산해 정산 금액이 1원 어긋난 경험. 한 건은 1원이지만 정산 대사에서는 「불일치」 한 줄이 된다.

2진수로는 0.1을 정확히 적을 수 없다



`double`은 2진 근삿값을, `BigDecimal`은 10진 숫자를 정수와 스케일(소수 자리 수)로 정확히 담는다.

> 이것이 이펙티브 자바 아이템 60: 정확한 답이 필요하다면 float와 double은 피하라.

BigDecimal은 문자열로 만든다: new BigDecimal(0.1)의 함정

`BigDecimal`을 쓰기만 하면 될까? 어떤 생성자로 만드는지가 중요하다.

```
System.out.println(new BigDecimal(0.1));
System.out.println(new BigDecimal("0.1"));
System.out.println(new BigDecimal("0.10").add(new BigDecimal("0.20")));
```

```
0.1000000000000000055511151231257827021181583404541015625
0.1
0.30
```

> `double`을 받는 생성자는 이미 오차가 낀 값을 그대로 옮긴다. 문자열로 만들고, `double`밖에 없으면 `BigDecimal.valueOf(0.1)`을 쓴다.

Money 값 객체 최소 구현: of, plus, times

호출부가 `BigDecimal`을 직접 다루지 않게 감싼다. Day 1의 불변 규칙 그대로다. equals/hashCode는 퀴즈 뒤에 붙인다. Money.java

```
import java.math.BigDecimal;
import java.math.RoundingMode;                    // percent()에서 쓴다

public final class Money {
    public static final Money ZERO = Money.of("0");
    private final BigDecimal amount;

    private Money(BigDecimal amount) {
        this.amount = amount;
    }

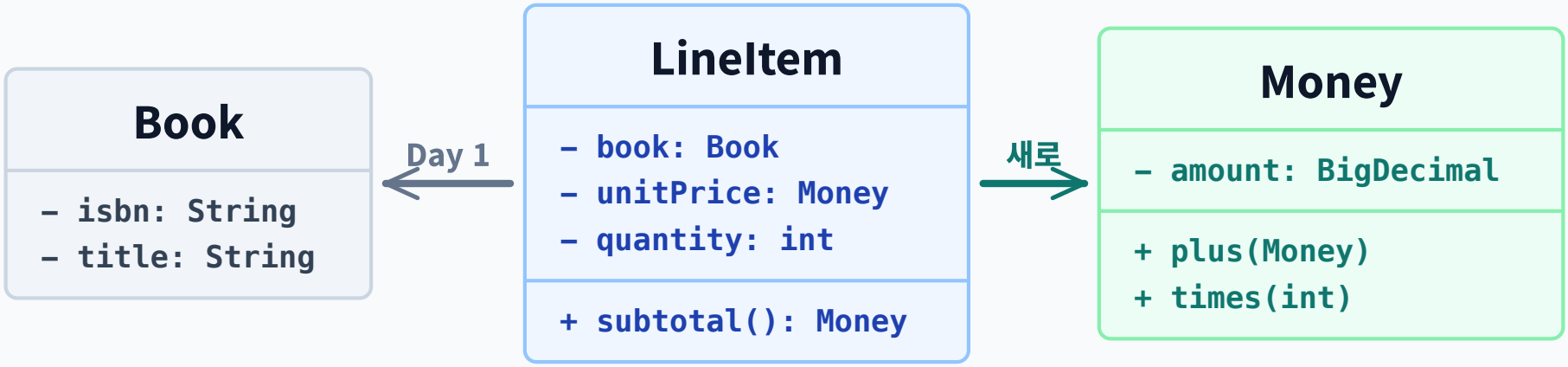
    public static Money of(String amount) {       // 문자열로만 받는다
        return new Money(new BigDecimal(amount));
    }

    public Money plus(Money other) {              // 새 Money를 돌려준다
        return new Money(amount.add(other.amount));
    }

    public Money times(int quantity) {
        return new Money(amount.multiply(BigDecimal.valueOf(quantity)));
    }

    @Override
    public String toString() {                    // "15000원"
        return amount.toPlainString() + "원";
    }
}
```

LineItem: 책, 단가, 수량



주문 항목 `LineItem`은 Day 1의 `Book`과 새 `Money`를 필드로 들고 있다(연관). 항목 금액을 계산하는 `subtotal()`도 `Money`로 돌려준다.

LineItem 코드: 항목 금액도 Money로 계산한다

LineItem.java

```
import java.util.Objects;

public final class LineItem {
    private final Book book;
    private final Money unitPrice;
    private final int quantity;

    public LineItem(Book book, Money unitPrice, int quantity) {
        if (quantity < 1) {
            throw new IllegalArgumentException("수량은 1 이상이어야 합니다.");
        }
        this.book = Objects.requireNonNull(book, "book");
        this.unitPrice = Objects.requireNonNull(unitPrice, "unitPrice");
        this.quantity = quantity;
    }

    public Book getBook() { return book; }
    public Money getUnitPrice() { return unitPrice; }
    public int getQuantity() { return quantity; }

    public Money subtotal() {
        return unitPrice.times(quantity);
    }
}
```

`Book`, `Money`가 모두 불변이라 `LineItem`도 불변이다. 방어적 복사가 필요 없다.

테스트: 소계를 더한 합계가 정확하다

LineItemTest.java — 실행하면 2건 통과.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class LineItemTest {
    private final Book objectBook = new Book("978-1", "객체지향");
    private final Book javaBook = new Book("978-2", "자바 설계");

    @Test
    @DisplayName("단가 6000원 책 2권의 소계는 12000원이다")
    void subtotalIsUnitPriceTimesQuantity() {
        LineItem item = new LineItem(javaBook, Money.of("6000"), 2);
        Money subtotal = item.subtotal();
        assertEquals("12000원", subtotal.toString());
    }

    @Test
    @DisplayName("두 항목의 소계를 더하면 합계 27000원이다")
    void totalOfTwoItems() {
        LineItem first = new LineItem(objectBook, Money.of("15000"), 1);
        LineItem second = new LineItem(javaBook, Money.of("6000"), 2);
        Money total = Money.ZERO.plus(first.subtotal()).plus(second.subtotal());
        assertEquals("27000원", total.toString());
    }
}
```

Money끼리 바로 assertEquals 하지 않고 toString을 비교했다. 이유는 퀴즈에서.

반올림은 코드에 적는다: percent와 RoundingMode.HALF_UP

원 단위로 자를 때 **어떻게** 자를지를 코드가 말한다. 앞에서 만든 `Money.java`에 `percent()`를 추가한다 (`RoundingMode` import는 이미 있다).

```
public Money percent(int percent) {
    BigDecimal result = amount.multiply(BigDecimal.valueOf(percent))
            .divide(BigDecimal.valueOf(100), 0, RoundingMode.HALF_UP);
    return new Money(result);
}
```

> 자릿수·반올림 없이 나누면 1/3 같은 몫에서 `ArithmeticException`이 난다.

테스트: 22000원의 70%는 정확히 15400원이다

MoneyTest.java — import는 퀴즈 뒤에 붙일 테스트 몫까지 미리 넣었다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.assertEquals;

class MoneyTest {
    @Test
    @DisplayName("정가 22000원의 70%는 15400원이다")
    void seventyPercentIsExact() {
        Money price = Money.of("22000");
        Money discounted = price.percent(70);
        assertEquals("15400원", discounted.toString());
    }

    @Test
    @DisplayName("1005원의 10%는 100.5원이라 반올림해 101원이다")
    void percentRoundsHalfUp() {
        Money price = Money.of("1005");
        Money tax = price.percent(10);
        assertEquals("101원", tax.toString());
    }
}
```

2건 통과. 앞에서 본 「30% 할인가가 1원 모자란다」의 1원이 사라졌다.

new BigDecimal("2.0").equals(new BigDecimal("2.00"))는? (퀴즈)

퀴즈

코드로 계산한 `1000`원과, DB의 `DECIMAL(10,2)` 컬럼에서 읽은 `1000.00`원을 비교하는 장면이다.

```
BigDecimal two = new BigDecimal("2.0");
BigDecimal twoPointZeroZero = new BigDecimal("2.00");

System.out.println(two.equals(twoPointZeroZero));
```

1. `true` — 같은 수다
2. `false`
3. 컴파일 에러

false — 그래서 Money.equals는 compareTo로 비교한다

정답

정답은 2번. `equals`는 스케일까지 보고 `compareTo`는 `0`이다. Day 1 8교시 「compareTo가 0이면 equals도 true」를 어기는 대표 예다. 앞에서 만든 `Money.java`에 두 메서드를 추가한다.

```
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (!(o instanceof Money)) {
        return false;
    }
    Money other = (Money) o;
    return amount.compareTo(other.amount) == 0;     // 스케일 무시
}

@Override
public int hashCode() {
    return amount.stripTrailingZeros().hashCode();  // 1000과 1000.00 → 같은 값
}
```

테스트: 1000원과 1000.00원은 같은 금액이다

앞에서 만든 `MoneyTest.java`의 끝(마지막 `}` 앞)에 두 테스트를 추가한다.

```
@Test
@DisplayName("1000원과 1000.00원은 같은 금액이다")
void equalsIgnoresScale() {
    Money computed = Money.of("1000");
    Money fromDb = Money.of("1000.00");
    assertEquals(computed, fromDb);
    assertEquals(computed.hashCode(), fromDb.hashCode());
}

@Test
@DisplayName("HashSet은 1000원과 1000.00원을 하나로 센다")
void hashSetCountsOnce() {
    Set<Money> amounts = new HashSet<>(Arrays.asList(Money.of("1000"), Money.of("1000.00")));
    assertEquals(1, amounts.size());
}
```

`MoneyTest` 4건 통과. 이제 `Money` 끼리 `assertEquals`로 바로 비교할 수 있다.

영수증 문자열을 += 로 쌓는다

주문 항목을 한 줄씩 적는 영수증. 가장 먼저 떠오르는 코드다. ReceiptPrinter.java

```
import java.util.List;

public final class ReceiptPrinter {
    private ReceiptPrinter() {
    }

    public static String print(List<LineItem> items) {
        String receipt = "";
        for (LineItem item : items) {
            receipt += item.getBook().getTitle() + " x" + item.getQuantity()
                    + " " + item.subtotal() + "\n";
        }
        return receipt;
    }
}
```

```
객체지향 x1 15000원
자바 설계 x2 12000원
```

현업에서 본 장면: 반복문에서 `+` 로 SQL·로그 문자열을 쌓던 코드.

테스트: 결과는 맞다

ReceiptPrinterTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ReceiptPrinterTest {
    private final List<LineItem> items = Arrays.asList(
            new LineItem(new Book("978-1", "객체지향"), Money.of("15000"), 1),
            new LineItem(new Book("978-2", "자바 설계"), Money.of("6000"), 2));

    @Test
    @DisplayName("한 줄에 제목, 수량, 소계를 적는다")
    void printsOneLinePerItem() {
        String receipt = ReceiptPrinter.print(items);
        assertEquals("객체지향 x1 15000원\n자바 설계 x2 12000원\n", receipt);
    }

    @Test
    @DisplayName("항목이 없으면 빈 문자열이다")
    void emptyItemsPrintNothing() {
        String receipt = ReceiptPrinter.print(Collections.<LineItem>emptyList());
        assertEquals("", receipt);
    }
}
```

2건 통과. 결과는 맞다. 그러면 무엇이 문제일까?

줄마다 지금까지의 문자열 전체를 다시 복사한다

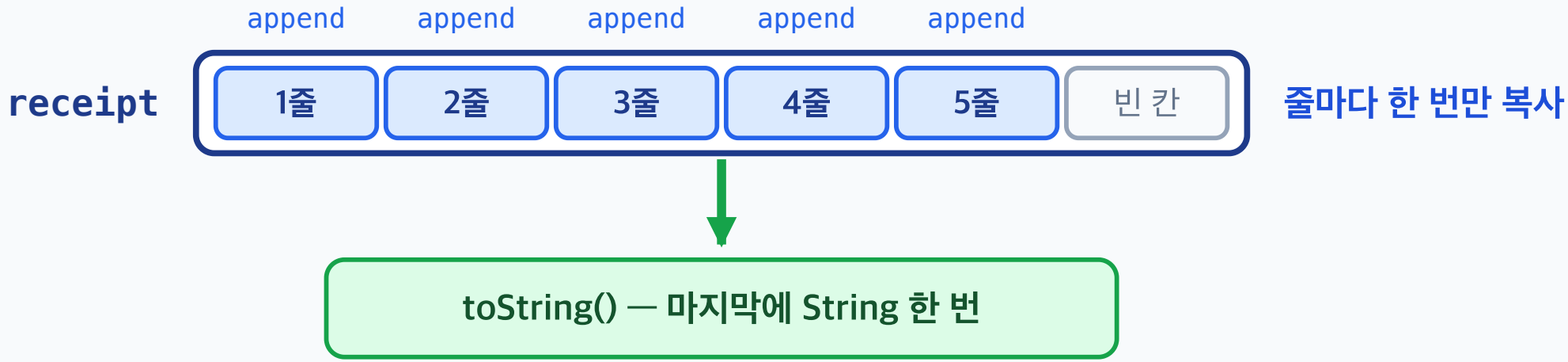


n번째 줄에서 n줄을 다시 복사 → 모두 합치면 약 $n^2/2$줄

`String`은 불변이라 `+=`마다 새 문자열을 만들고 앞 내용을 통째로 옮긴다. 강사 PC 측정값(실행마다 다름):

```
10000줄: += 397ms, StringBuilder 0ms
20000줄: += 952ms, StringBuilder 0ms
40000줄: += 3132ms, StringBuilder 0ms
```

버퍼 하나에 이어 붙인다: StringBuilder



`StringBuilder`는 늘어나는 버퍼 하나에 줄을 한 번씩만 붙이고, 끝에 `toString()`으로 `String`을 한 번 만든다. 칸이 모자라면 버퍼를 넉넉히 키운다.

> 이것이 이펙티브 자바 아이템 63: 문자열 연결은 느리니 주의하라.

ReceiptPrinter 구현: += 를 append 체인으로

앞에서 만든 `ReceiptPrinter.java`의 `print()`를 이것으로 바꾼다. `+=` 한 줄이 `append` 체인으로 바뀐다. `append(item.subtotal())`은 `Money.toString()`을 붙인다.

```
public static String print(List<LineItem> items) {
    StringBuilder receipt = new StringBuilder();
    for (LineItem item : items) {
        receipt.append(item.getBook().getTitle())
                .append(" x").append(item.getQuantity())
                .append(' ').append(item.subtotal())
                .append('\n');
    }
    return receipt.toString();
}
```

`ReceiptPrinterTest` 2건이 그대로 통과한다. 결과는 같고 복사량만 줄었다.

한 줄짜리 +는 괜찮다: 언제 쓰고 언제 안 쓰나

장면	쓰는 것	이유
한 식 안의 연결 `"합계 " + total`	`+`	컴파일러가 버퍼 하나로 바꿔 준다. 읽기 쉽다
반복문 안에서 누적	`StringBuilder`	`+=` 는 반복마다 전체를 다시 복사한다
구분자로 잇기	`String.join` , `Collectors.joining`	Java 8에 있다. 마지막 구분자 처리가 필요 없다
몇 번 안 되는 고정 연결	읽기 쉬운 쪽	차이가 측정되지 않는다

> 「그럼 모든 `+` 를 `StringBuilder` 로 바꿔야 하나요?」 아니다. 기준은 「반복하면서 쌓는가」 하나다.

영수증 끝에 합계 줄을 붙인다 (실습)

실습

파일 수정: 앞에서 만든 `ReceiptPrinter.java`를 고친다. 먼저 `ReceiptPrinterTest.java`의 두 테스트를 지우고 아래 두 건으로 바꿔 친다. 줄 단위로 비교하니 합계 줄이 붙어도 항목 줄 검사는 깨지지 않는다.

```
@Test
@DisplayName("항목 줄은 제목, 수량, 소계 순서다")
void itemLines() {
    String[] lines = ReceiptPrinter.print(items).split("\n");
    assertEquals("객체지향 x1 15000원", lines[0]);
    assertEquals("자바 설계 x2 12000원", lines[1]);
}

@Test
@DisplayName("마지막 줄은 '합계 27000원'이다")
void lastLineIsTotal() {
    String[] lines = ReceiptPrinter.print(items).split("\n");
    assertEquals("합계 27000원", lines[lines.length - 1]);
}
```

지금은 두 번째 테스트가 `expected: <합계 27000원> but was: <자바 설계 x2 12000원>`으로 실패한다.

실습 힌트: 반복문 안에서 더하고, 뒤에서 한 줄 붙인다

반복문에서 소계를 더하고, 반복문 뒤에 한 줄을 붙인다(기존 `append` 체인은 그대로). 발췌:

```
Money total = Money.ZERO;
for (LineItem item : items) {
    // ... 기존 append 체인 그대로
    total = total.plus(item.subtotal());
}
receipt.append("합계 ").append(total).append('\n');
```

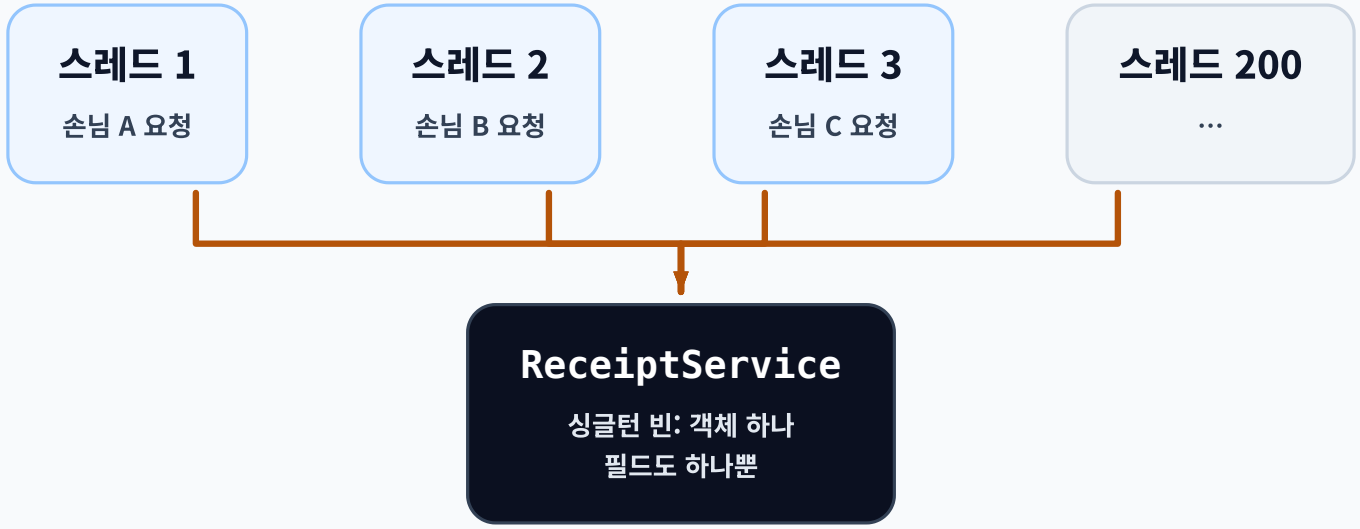
`ReceiptPrinterTest`를 실행해 2건 통과를 확인한다. `Money`가 불변이라 `total = total.plus(...)`로 다시 대입한다 — Day 1의 불변 `Health`와 같은 모양이다.

8교시

스프링 `@Service`는 하나다

요청 스레드 수백 개가 같은 객체를 쓴다. 그 객체에 요청마다 바뀌는 필드가 있다면?

요청 스레드 수백 개가 빈 하나로 모인다



톰캣은 요청마다 스레드풀의 스레드 하나를 쓴다(기본 최대 200개). 그런데 `@Service` 빈은 객체 하나라서, 모든 스레드가 **같은 객체의 같은 필드**를 본다.

> 현업 장면: 빈 필드에 「현재 사용자」를 담았다가 남의 영수증에 내 이름이 찍힌 버그.

ReceiptService: 빈 필드에 요청 상태를 담았다

ReceiptService.java — 손님 이름과 책을 필드에 두고 `begin` → `add` → `finish` 순서로 쓴다(7교시 `LineItem`, `Money`).

```
import java.util.ArrayList;
import java.util.List;

public class ReceiptService {
    private String customer;
    private final List<LineItem> items = new ArrayList<>();

    public void begin(String customer) {
        this.customer = customer;
        items.clear();
    }

    public void add(LineItem item) {
        items.add(item);
    }

    public String finish() {
        Money total = Money.ZERO;
        for (LineItem item : items) {
            total = total.plus(item.subtotal());
        }
        return customer + " 손님 합계 " + total;
    }
}
```

테스트: 두 손님의 호출이 엇갈리면 영수증이 섞인다

ReceiptServiceTest.java — 스레드 없이 호출 순서만 엇갈린다(실제로는 요청 스레드가 만드는 순서). 매번 똑같이 깨진다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ReceiptServiceTest {

    private final ReceiptService service = new ReceiptService();   // 객체 하나
    private final Book objectBook = new Book("978-1", "객체지향");
    private final Book javaBook = new Book("978-2", "자바 설계");

    @Test
    @DisplayName("두 손님의 호출이 엇갈려도 영수증은 각자 것이다")
    void interleavedRequests() {
        service.begin("A");                                    // 손님 A 요청
        service.begin("B");                                    // 손님 B 요청
        service.add(new LineItem(objectBook, Money.of("15000"), 1)); // A
        service.add(new LineItem(javaBook, Money.of("6000"), 2));    // B

        assertEquals("A 손님 합계 15000원", service.finish());      // A
        assertEquals("B 손님 합계 12000원", service.finish());      // B
    }
}
```

```
✘ expected: <A 손님 합계 15000원> but was: <B 손님 합계 27000원>
```

처방 ①: 필드를 지우고 요청 상태는 파라미터로 받는다

앞에서 만든 `ReceiptService.java` 를 이것으로 바꾼다.

```
import java.util.List;

public class ReceiptService {

    public String receipt(String customer, List<LineItem> items) {
        Money total = Money.ZERO;
        for (LineItem item : items) {
            total = total.plus(item.subtotal());
        }
        return customer + " 손님 합계 " + total;
    }
}
```

필드가 없으니 객체 하나를 몇 스레드가 함께 써도 서로 건드릴 것이 없다. **잠금도 필요 없다.**

`ReceiptServiceTest.java` 의 테스트 메서드를 이것으로 바꾸고 `import java.util.ArrayList;` , `import java.util.List;` 를 더한다.

```
@Test
@DisplayName("두 손님의 호출이 엇갈려도 영수증은 각자 것이다")
void interleavedRequests() {
    List<LineItem> a = new ArrayList<>();   // 손님 A 요청의 지역 변수
    List<LineItem> b = new ArrayList<>();   // 손님 B 요청의 지역 변수
    a.add(new LineItem(objectBook, Money.of("15000"), 1));
    b.add(new LineItem(javaBook, Money.of("6000"), 2));

    assertEquals("A 손님 합계 15000원", service.receipt("A", a));
    assertEquals("B 손님 합계 12000원", service.receipt("B", b));
}
```

1건 통과. 지역 변수는 스레드마다 따로 생긴다.

공유 상태 처방 사다리: 위에서부터 고른다

단계	처방	예	아이템
1	**공유하지 않는다**: 무상태, 요청 상태는 지역 변수·파라미터로	`ReceiptService`	—
2	**불변 객체만 공유한다**: 바뀌지 않으면 함께 써도 안전	`Money`, `LineItem`, `DateTimeFormatter`	17
3	**꼭 바뀌는 상태를 공유해야 하면** 동기화하되 짧게	다음에 볼 `Order`	78, 79

대부분의 빈은 1단계로 끝난다. 3단계는 정말 공유되는 상태가 있을 때만 쓴다. 이것이 이펙티브 자바 아이템 17(변경 가능성을 최소화하라)이 동시성에서 주는 이득이다.

> 이어지는 `Order`는 3단계 사례다. 같은 사용자가 여러 탭에서 동시에 담는 세션 범위 장바구니처럼, 정말 공유되는 주문이다.

처방 ③의 사례 Order: 여러 탭이 같은 주문에 책을 담는다

Order.java — 세션 범위 장바구니처럼 여러 요청이 정말 같은 주문 하나에 담는다. 불변 `LineItem`, `Money`를 그대로 쓴다.

```
import java.util.ArrayList;
import java.util.List;

public class Order {
    private final List<LineItem> items = new ArrayList<>();

    public void add(LineItem item) {
        items.add(item);
    }

    public List<LineItem> getItems() {
        return items;
    }

    public Money total() {
        Money total = Money.ZERO;
        for (LineItem item : items) {
            total = total.plus(item.subtotal());
        }
        return total;
    }
}
```

두 스레드가 1만 개씩 담으면 2만 개가 안 된다

OrderRaceDemo.java — 같은 `order`에 두 스레드가 1만 번씩 `add` 한다.

```
public class OrderRaceDemo {
    public static void main(String[] args) throws InterruptedException {
        Order order = new Order();
        LineItem item = new LineItem(new Book("978-1", "객체지향"), Money.of("15000"), 1);

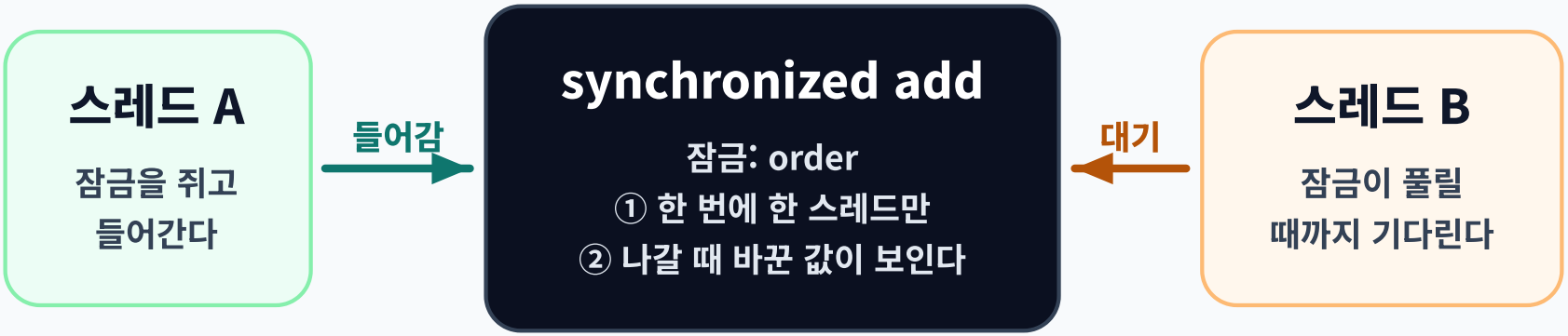
        Runnable task = () -> {
            for (int i = 0; i < 10_000; i++) {
                order.add(item);
            }
        };
        Thread a = new Thread(task);
        Thread b = new Thread(task);
        a.start(); b.start();
        a.join(); b.join();

        System.out.println("항목 수: " + order.getItems().size());
    }
}
```

```
항목 수: 13124        ← 실행 예(매번 다름)
Exception in thread "Thread-0" java.lang.ArrayIndexOutOfBoundsException ...
항목 수: 10010
```

`add`는 size 읽기 → 쓰기 → size 늘리기 여러 단계라, 끼어들면 한 건이 덮어써지거나 예외가 난다.

한 번에 한 스레드만 들어오게 한다: synchronized



`add` 하나를 끼어들 수 없는 한 덩어리로 만든다. `synchronized`는 두 가지를 준다. **배타적 실행**(한 번에 하나)과 **보이기**(앞 스레드가 바꾼 값을 다음 스레드가 본다).

> 이것이 이펙티브 자바 아이템 78: 공유 중인 가변 데이터는 동기화해 사용하라.

synchronized add 구현

앞에서 만든 Order.java 를 고친다 — `add()` 와 `total()` 선언에 `synchronized` 를 붙인다. 목록을 바꾸는 `add` 와 목록을 읽는 `total` 이 **같은 잠금**(`this`)을 쓴다.

```
public synchronized void add(LineItem item) {
    items.add(item);
}

public synchronized Money total() {
    Money total = Money.ZERO;
    for (LineItem item : items) {
        total = total.plus(item.subtotal());
    }
    return total;
}
```

`add` 만 잠그고 `total` 을 그냥 두면, 합계를 세는 도중에 다른 스레드가 목록을 바꿀 수 있다. 쓰는 쪽과 읽는 쪽을 함께 잠근다.

테스트: 두 스레드 × 10,000 → 정확히 20,000개, 합계 3억 원

OrderConcurrencyTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class OrderConcurrencyTest {

    @Test
    @DisplayName("두 스레드가 1만 개씩 담으면 정확히 2만 개, 합계 3억 원이다")
    void twoThreadsAddTwentyThousand() throws InterruptedException {
        Order order = new Order();
        LineItem item = new LineItem(new Book("978-1", "객체지향"), Money.of("15000"), 1);
        Runnable task = () -> {
            for (int i = 0; i < 10_000; i++) {
                order.add(item);
            }
        };
        Thread a = new Thread(task);
        Thread b = new Thread(task);
        a.start(); b.start();
        a.join(); b.join();

        assertEquals(20_000, order.getItems().size());
        assertEquals(Money.of("300000000"), order.total());
    }
}
```

1건 통과. 통과 한 번이 「경쟁이 없다」는 증명은 아니다. 동기화는 설계로 먼저 정한다.

읽는 쪽도 동기화해야 한다: 멈추지 않는 스레드 (퀴즈)

1초 뒤 `main`이 플래그를 켠다. 일꾼 스레드는 멈출까?

```
private static boolean stopRequested;

Thread worker = new Thread(() -> {
    int count = 0;
    while (!stopRequested) {
        count++;
    }
    System.out.println("멈춤: " + count);
});
worker.start();
TimeUnit.SECONDS.sleep(1);
stopRequested = true;   // 일꾼은 이 값을 끝내 못 볼 수 있다
```

동기화가 없으면 JVM은 이 반복문을 `while (true)`로 바꿔도 된다(강의에서는 실행하지 않는다).

> 구두 질문: `volatile`로 충분한 것은 플래그, `count++`, `add` 중 무엇? 답은 다음 장에서 본다.

정답: 멈추지 않을 수 있다. `volatile` 한 단어로 고친다

VolatileStopFlagDemo.java

```
private static volatile boolean stopRequested;   // 쓰면 다른 스레드가 바로 본다

Thread worker = new Thread(() -> {
    int count = 0;
    while (!stopRequested) {
        count++;
    }
    System.out.println("멈춤");
});
worker.start();

TimeUnit.SECONDS.sleep(1);
stopRequested = true;
worker.join();
```

- 동기화가 없으면 `main` 이 쓴 값을 일꾼이 본다는 보장이 없다. 그래서 멈추지 않을 수 있다.
- `volatile` 을 붙이면 1초 뒤 `멈춤` 이 찍힌다. 한쪽만 쓰고 다른 쪽은 읽기만 하는 **플래그**에 충분하다.
- `volatile` 은 「보이게」 만 한다. `count++` · `add` 처럼 읽고-바꾸고-쓰는 연산은 여전히 깨지므로 `synchronized` · `AtomicInteger` 를 쓴다.

요구 추가: 담을 때마다 합계와 함께 저장한다

새 요구 때문에 `Order`가 저장소를 받는다. 저장소는 메서드 하나짜리 인터페이스다(실제로는 DB나 원격 API).

OrderRepository.java

```
import java.util.List;

public interface OrderRepository {
    void save(List<LineItem> items, Money total);
}
```

앞에서 만든 Order.java를 고친다 — 여러 곳이 바뀌니 파일 전체를 이것으로 바꾼다(다음 장까지). 기본 생성자 `Order()`가 없어지므로 앞 장면 확인용 OrderRaceDemo.java, OrderConcurrencyTest.java는 지운다.

```
import java.util.ArrayList;
import java.util.List;

public class Order {
    private final List<LineItem> items = new ArrayList<>();
    private final OrderRepository repository;

    public Order(OrderRepository repository) {
        this.repository = repository;
    }
```

배운 대로 add를 통째로 잠근다

Order.java (이어서) — 합계 반복문은 `private static totalOf`로 옮겼다.

```
    public synchronized void add(LineItem item) {
        items.add(item);
        Money total = total();
        repository.save(items, total);   // 느린 저장도 잠금 안에서
    }

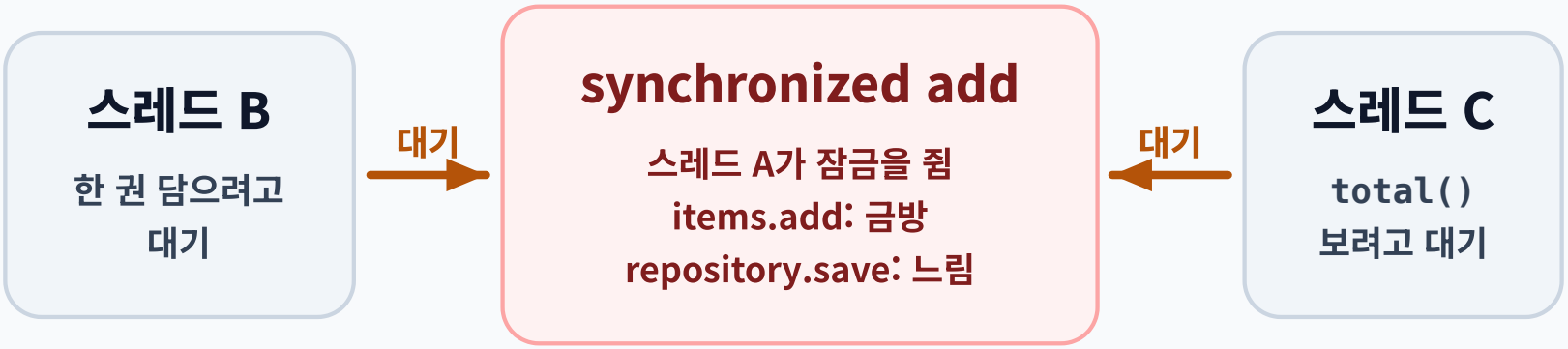
    public List<LineItem> getItems() {
        return items;
    }

    public synchronized Money total() {
        return totalOf(items);
    }

    private static Money totalOf(List<LineItem> lineItems) {
        Money total = Money.ZERO;
        for (LineItem item : lineItems) {
            total = total.plus(item.subtotal());
        }
        return total;
    }
}
```

결과는 맞는다. 무엇이 문제일까?

저장이 느리면 모든 스레드가 줄을 선다



스레드 A가 저장을 기다리는 동안 잠금을 놓지 않는다. 한 권 담기, 합계 보기처럼 금방 끝날 일도 저장 시간만큼 막힌다.

> 현업 장면: `synchronized` 메서드 안에서 DB를 호출해, 트래픽이 몰리자 요청 스레드가 줄줄이 대기한 사례.

가짜 저장소가 잠금을 지켜본다: RecordingOrderRepository

`RecordingOrderRepository.java` — 느린 DB 대신 「저장하는 순간 주문 잠금을 쥐고 있었나」를 기록한다. Mockito 없이 직접 만든다.

```
import java.util.List;

class RecordingOrderRepository implements OrderRepository {
    private Object lock;
    private boolean lockHeldDuringSave;
    private int saveCount;

    void watch(Object lock) {
        this.lock = lock;
    }

    @Override
    public synchronized void save(List<LineItem> items, Money total) {
        if (lock != null && Thread.holdsLock(lock)) {   // 지금 이 스레드가 lock을 쥐고 있나
            lockHeldDuringSave = true;
        }
        saveCount++;
    }

    synchronized boolean wasLockHeldDuringSave() {
        return lockHeldDuringSave;
    }

    synchronized int saveCount() {
        return saveCount;
    }
}
```

테스트로 드러낸다: 저장하는 동안에도 잠금을 쥐고 있다

OrderLockTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;

class OrderLockTest {

    private final Book objectBook = new Book("978-1", "객체지향");

    @Test
    @DisplayName("저장하는 동안에는 주문 잠금을 쥐고 있지 않는다")
    void saveRunsOutsideLock() {
        RecordingOrderRepository repository = new RecordingOrderRepository();
        Order order = new Order(repository);
        repository.watch(order);

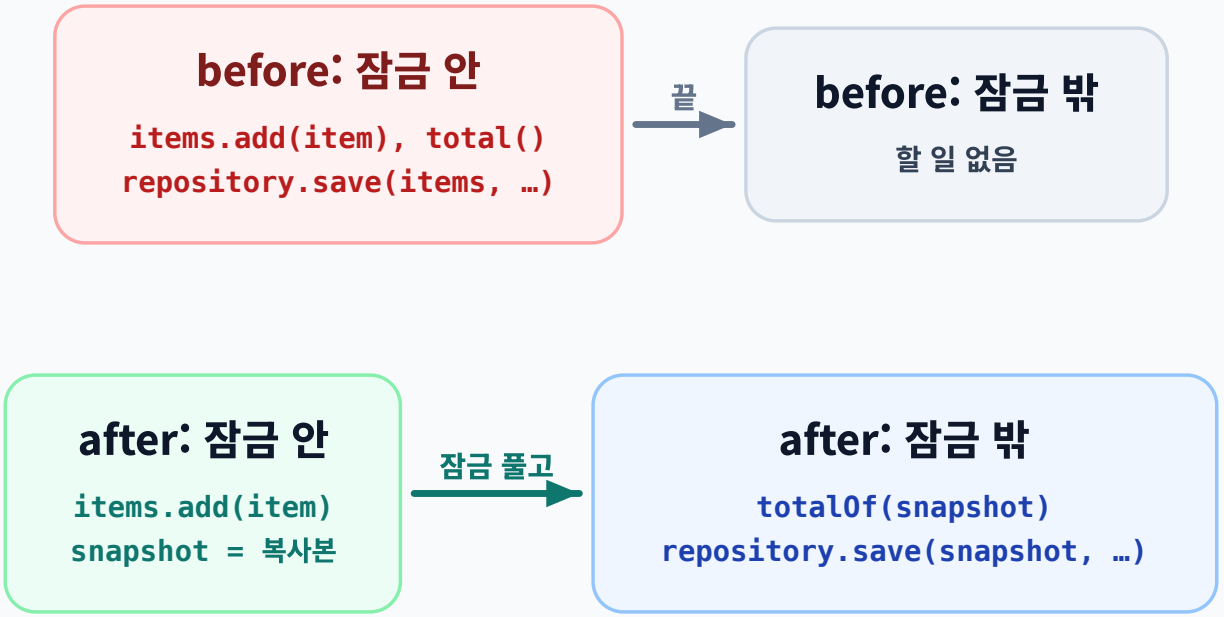
        order.add(new LineItem(objectBook, Money.of("15000"), 1));

        assertFalse(repository.wasLockHeldDuringSave());
        assertEquals(1, repository.saveCount());   // 결과(저장 1번)는 맞다
    }
}
```

```
✘ 저장하는 동안에는 주문 잠금을 쥐고 있지 않는다
    expected: <false> but was: <true>
```

저장 횟수 같은 결과는 맞는다. 틀린 것은 잠금 구간이다.

잠금 안에서는 공유 상태만, 나머지는 밖에서



잠금 안에서는 공유 목록 변경과 **복사본(스냅샷)** 뜨기만, 계산·저장은 스냅샷으로 잠금 밖에서.

> 이것이 이펙티브 자바 아이템 79: 과도한 동기화는 피하라.

스냅샷 구현: 잠금 안에서 복사, 밖에서 계산·저장

앞에서 만든 Order.java 의 `add()` 를 이것으로 바꾼다 — 메서드 전체 대신 `synchronized (this)` 블록만 잠근다.

`import java.util.Collections;` 도 더한다.

```
public void add(LineItem item) {
    List<LineItem> snapshot;
    synchronized (this) {
        items.add(item);
        snapshot = Collections.unmodifiableList(new ArrayList<>(items));
    }
    Money total = totalOf(snapshot);   // 잠금 밖
    repository.save(snapshot, total);  // 잠금 밖
}
```

합계는 앞에서 만든 `totalOf` 에 스냅샷을 넘겨 센다. `total()` 은 그대로 `synchronized` 다.

잠금 대상은 테스트에서 `holdsLock(order)` 로 보려고 `this` 로 두었다. 실무에서는 바깥에서 건드릴 수 없는 `private final Object lock` 이 더 안전하다.

동시에 담아도 저장 횟수가 맞는지 확인한다

앞에서 만든 `OrderLockTest.java`의 끝(마지막 `}` 앞)에 테스트를 하나 더한다.

```
    @Test
    @DisplayName("두 스레드가 1000개씩 담아도 항목 2000개, 저장 2000번이다")
    void concurrentAddsWithSnapshot() throws InterruptedException {
        RecordingOrderRepository repository = new RecordingOrderRepository();
        Order order = new Order(repository);
        LineItem item = new LineItem(objectBook, Money.of("15000"), 1);
        Runnable task = () -> {
            for (int i = 0; i < 1_000; i++) {
                order.add(item);
            }
        };
        Thread a = new Thread(task);
        Thread b = new Thread(task);

        a.start(); b.start();
        a.join(); b.join();

        assertEquals(2_000, order.getItems().size());
        assertEquals(2_000, repository.saveCount());
    }
```

테스트 통과: 불변 LineItem·Money라서 스냅샷을 그대로 넘겨도 안전하다

`OrderLockTest` 2건 통과.

테스트	결과
저장하는 동안에는 주문 잠금을 쥐고 있지 않는다	통과 (스냅샷 전에는 실패)
두 스레드가 1000개씩 담아도 항목 2000개, 저장 2000번이다	통과

스냅샷은 목록만 복사한다. 그 안의 `LineItem`, `Money`, `Book`은 바뀌지 않는 객체라 저장소와 나눠 가져도 안전하다. 불변 객체가 동기화할 거리를 줄여 준다.

> Q&A: 잠금 밖에서 저장하면 **저장 순서**는 보장되지 않는다. 늦게 뜬 스냅샷 [책1, 책2]가 먼저 저장되고, 먼저 뜬 [책1]이 나중에 덮어쓸 수 있다. 순서까지 지켜야 하면 스냅샷에 버전 번호를 붙여 오래된 것을 버리거나, 저장을 한 스레드에 줄 세운다.

처방 ②: 불변이면 static final로 함께 써도 된다

읽기용 비교다(따라 치지 않는다). 날짜 유틸 `OrderDates.java`의 필드와 메서드만 보인다.

레거시: `SimpleDateFormat`

```
private static final SimpleDateFormat FORMAT =
        new SimpleDateFormat("yyyy-MM-dd");

public static String format(Date date) {
    return FORMAT.format(date);   // 안의 달력을 고쳐 쓴다
}
```

`SimpleDateFormat`은 가변이다. 여러 요청이 동시에 쓰면 **가끔** 남의 날짜가 찍히거나 예외가 난다. 재현이 안 돼서 더 무섭다.

Java 8: `DateTimeFormatter`

```
private static final DateTimeFormatter FORMAT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd");

public static String format(LocalDate date) {
    return FORMAT.format(date);   // 바뀌는 필드가 없다
}
```

`DateTimeFormatter`는 불변이다. 잠금 없이 모든 스레드가 하나를 함께 써도 안전하다.

> 불변 객체 = 동기화가 필요 없는 객체. `Money`, `LineItem`도 같은 이유로 스냅샷에 담아 넘길 수 있었다.

getItems가 내부 목록 대신 스냅샷을 돌려주게 한다 (실습)

실습 앞에서 만든 `Order.java`의 `getItems()`를 고쳐 `OrderTest.java`를 초록으로 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class OrderTest {

    @Test
    @DisplayName("getItems로 받은 목록은 수정할 수 없고 주문 항목 수는 그대로다")
    void getItemsReturnsSnapshot() {
        Order order = new Order((items, total) -> { });
        order.add(new LineItem(new Book("978-1", "객체지향"), Money.of("15000"), 1));
        order.add(new LineItem(new Book("978-2", "자바 설계"), Money.of("6000"), 2));

        List<LineItem> items = order.getItems();

        assertThrows(UnsupportedOperationException.class, () -> items.clear());
        assertEquals(2, order.getItems().size());
    }
}
```

```
Expected java.lang.UnsupportedOperationException to be thrown, but nothing was thrown.
```

실습 풀이: getItems도 잠금 안에서 복사한다

Order.java 의 `getItems()` 를 이것으로 바꾼다.

```
public List<LineItem> getItems() {
    synchronized (this) {
        return Collections.unmodifiableList(new ArrayList<>(items));
    }
}
```

`OrderTest` 를 실행해 1건 통과를 확인한다. 테스트의 저장소 `(items, total) -> { }` 는 람다다(`OrderRepository` 가 메서드 하나짜리 인터페이스라서).

왜: 복사도 잠금 안에서 해야 다른 스레드의 `add` 와 섞이지 않는다. 방어적 복사(아이템 50)와 동기화(아이템 78)가 한 줄에서 만난다.

Day 2 복습: 문제 → 처방 → 아이템

교시	겪은 문제	처방	아이템
1	꺼낼 때 터지는 목록	Raw 타입 대신 제네릭, 배열보다 리스트	26, 28
2	`List<Marine>`을 못 받는 유틸	읽기 전용 `? extends`, 쓰기 전용 `? super`	31
3	경고 없는 유틸과 타입별 보관함	제네릭 메서드, 타입 안전 이종 컨테이너	30, 33
4	조건 하나에 여섯 줄	람다, 표준 함수형 인터페이스	42, 44
5	스트림처럼 생긴 반복문	부작용 없는 스트림, 병렬은 신중히	45, 46, 48
6	`if (list != null)`이 퍼진 코드	빈 컬렉션, `Optional` 반환	54, 55
7	1원이 어긋난 정산과 영수증	`BigDecimal`(`Money`), `StringBuilder`	60, 63
8	요청이 섞인 영수증, 동시에 담긴 주문	공유하지 않기 → 불변만 공유 → 짧은 잠금 + 스냅샷	17, 78, 79

최종 연습문제: Squad에 두 질문을 더한다

숙제 수업에서는 소개만 하고 집에서 푼다. 풀이는 별도로 공유한다.

6교시에 만든 `Squad.java` 에 아래 import 한 줄과 두 메서드를 더한다. 처음에는 예외만 던진다.

```
import java.util.Map;

    Map<UnitType, Long> countByType() {          // 종류별 유닛 수
        throw new UnsupportedOperationException("실습: countByType");
    }

    Optional<Unit> strongest() {                 // 체력이 가장 높은 유닛, 없으면 빈 Optional
        throw new UnsupportedOperationException("실습: strongest");
    }
```

실패하는 테스트(다음 장)	조립할 것
종류별 개수를 센다: 보병 2, 일꾼 1, 차량 1, 공중 1	스트림 + `groupingBy` + `counting` (5교시)
가장 체력이 높은 유닛은 체력 120인 Wraith다	`max` + `Comparator.comparingInt` , 메서드 참조 (5교시 퀴즈 풀이)
빈 부대의 strongest는 빈 Optional이다	`null` 대신 `Optional` (6교시)

반복문 없이 각 메서드가 한 문장이면 된다.

숙제 테스트: 부대 구성이 결과를 정한다

SquadFinalTest.java — 탱크는 `SiegeTank.ofDamaged(100)`으로 넣는다. `create()`(체력 150)로 넣으면 탱크가 가장 강해진다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.Map;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;

class SquadFinalTest {

    private Squad mixedSquad() {
        Squad squad = new Squad();
        squad.add(Marine.create());
        squad.add(Marine.create());
        squad.add(SCV.create());
        squad.add(SiegeTank.ofDamaged(100));   // create()가 아니다: 체력 100
        squad.add(Wraith.create());            // 체력 120, 이 부대에서 가장 높다
        return squad;
    }
```

숙제 테스트: 세 질문

SquadFinalTest.java (이어서) — 처음에는 3건 모두 `UnsupportedOperationException`으로 실패한다.

```
    @Test
    @DisplayName("종류별 개수를 센다: 보병 2, 일꾼 1, 차량 1, 공중 1")
    void countsByType() {
        Map<UnitType, Long> counts = mixedSquad().countByType();
        assertEquals(Long.valueOf(2), counts.get(UnitType.INFANTRY));
        assertEquals(Long.valueOf(1), counts.get(UnitType.WORKER));
        assertEquals(Long.valueOf(1), counts.get(UnitType.VEHICLE));
        assertEquals(Long.valueOf(1), counts.get(UnitType.AIRCRAFT));
    }

    @Test
    @DisplayName("가장 체력이 높은 유닛은 체력 120인 Wraith다")
    void strongestIsHighestHp() {
        Optional<Unit> strongest = mixedSquad().strongest();
        assertEquals("Wraith", strongest.map(Unit::getName).orElse("없음"));
    }

    @Test
    @DisplayName("빈 부대의 strongest는 빈 Optional이다")
    void strongestOfEmptySquadIsEmpty() {
        Optional<Unit> strongest = new Squad().strongest();
        assertFalse(strongest.isPresent());
    }
}
```

이틀을 마치며: 점검 질문

1. 우리 서비스의 싱글턴 객체에 **바뀌는 필드**가 있는가? 있다면 누가 그 필드를 잠그고, 잠금 안에서 DB·원격 호출을 하고 있지는 않은가?
2. 오늘 만든 `Money`, `LineItem`처럼 불변으로 바꾸면 동기화가 필요 없어지는 클래스를 하나 골라 보자.
3. 어제와 오늘 복습 표의 문제 중 지난달 우리 코드에서 본 것 하나와, 내일 바꿀 한 줄을 말해 보자.

> 이틀 동안 실수를 테스트가 잡게 했고(Day 1), 컴파일러가 잡게 했고, 같은 일을 더 짧고 정확하게 썼다(Day 2). 스프링이 없어도 남는 것은 이 설계 감각이다.

Q&A

이틀 동안 수고 많으셨습니다. 궁금한 점을 질문해 주세요.

수업 뒤 질문: junsuk.oh@survivalcoding.com